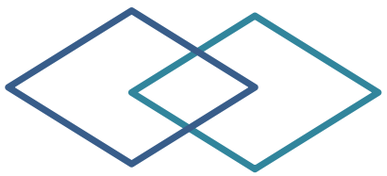


第12章

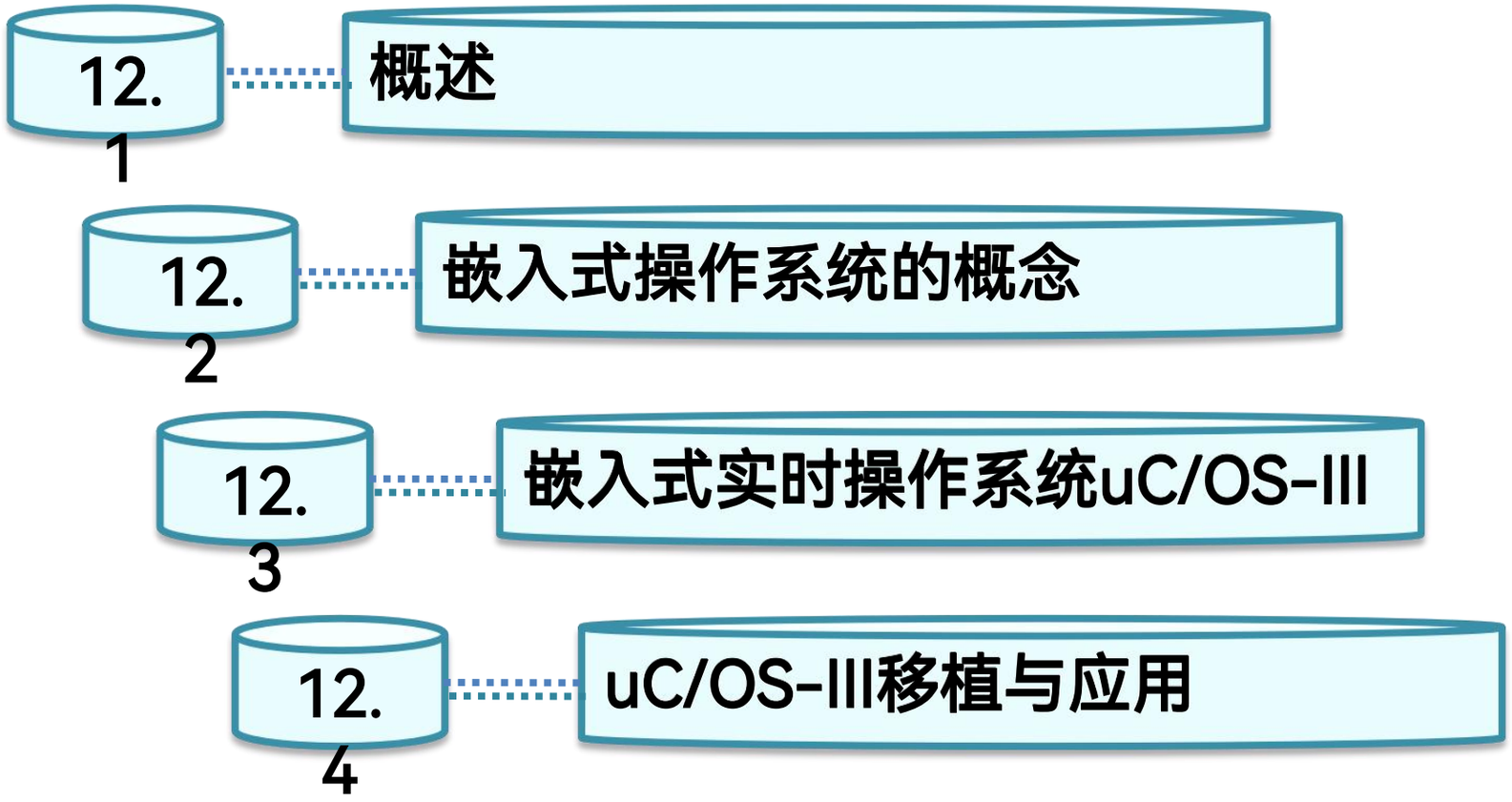
嵌入式操作系统



本章知识与能力要求

- 理解和掌握操作系统基本概念和功能；
- 理解和掌握uC/OS-III的任务、任务组成和任务优先级；
- 理解和掌握uC/OS-III的时钟和中断；
- 掌握将uC/OS-III移植到STM32F103系列微控制器的方法；
- 掌握基于uC/OS-III的多任务应用。

第12章 嵌入式操作系统



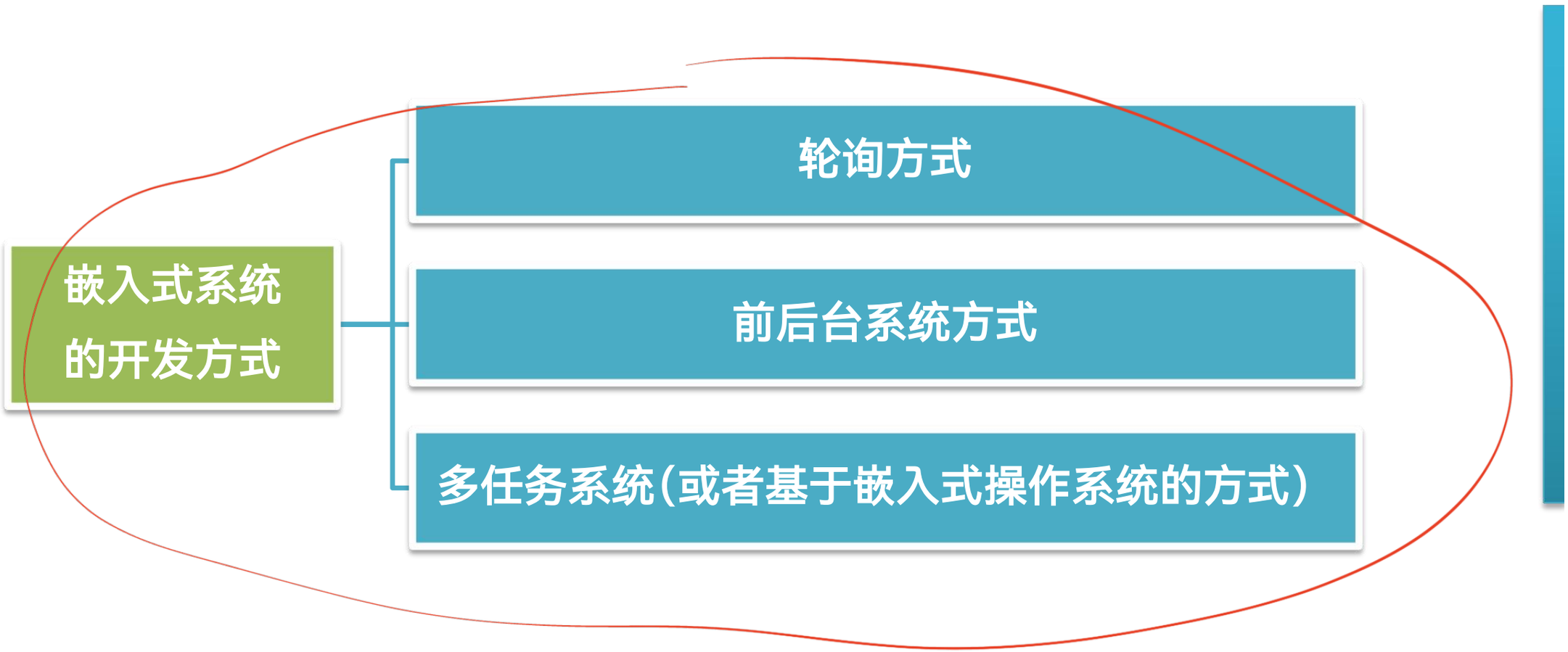
12.1 概述



12.1.1 嵌入式系统的开发方式

12.1.2 计算机系统的分层设计思想

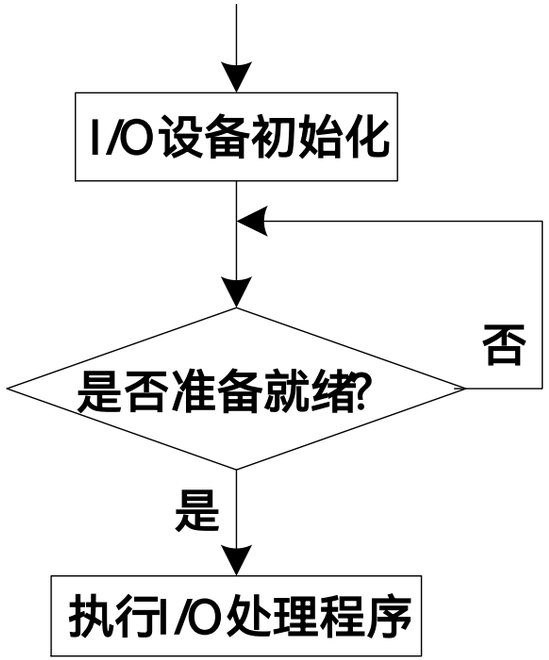
12.1.1 嵌入式系统的开发方式



12.1.1 嵌入式系统的开发方式

轮询方式

CPU通过程序不断查询I/O设备是否准备就绪，效率较低。



按键控制LED灯亮灭，程序代码如下：

```
int main(void)
{
    LED_Init(); //LED初始化
    KEY_Init();  //按键S1初始化
    while(1)
    {   if( !S1)   //查询按键S1是否按下
        {Delay(1000000);  //延迟一段时间

GPIO_ResetBits(GPIOB,GPIO_Pin_5);
                //点亮LED灯
        }
    }
}
```

12.1.1 嵌入式系统的开发方式

前后台系统方式

- 通常由一个大循环和若干中断组成，**大循环是后台**，一般由main()函数中的while(1)死循环构成，**中断是前台**，由各种中断服务程序组成。
- 前后台方式程序流程比较简单，常应用于单片机裸机应用程序的开发。

按键控制LED灯亮灭

```
int main(void)
{
   LED_Init();  //LED初始化
   GPIO_ResetBits(GPIOB,GPIO_Pin_5);
   EXTI_Key_Init(); //按键外部中断初始化
   while(1)   { }
}
void EXTI3_IRQHandler(void)
{
   if( EXTI_GetITStatus(EXTI_Line3) != RESET)

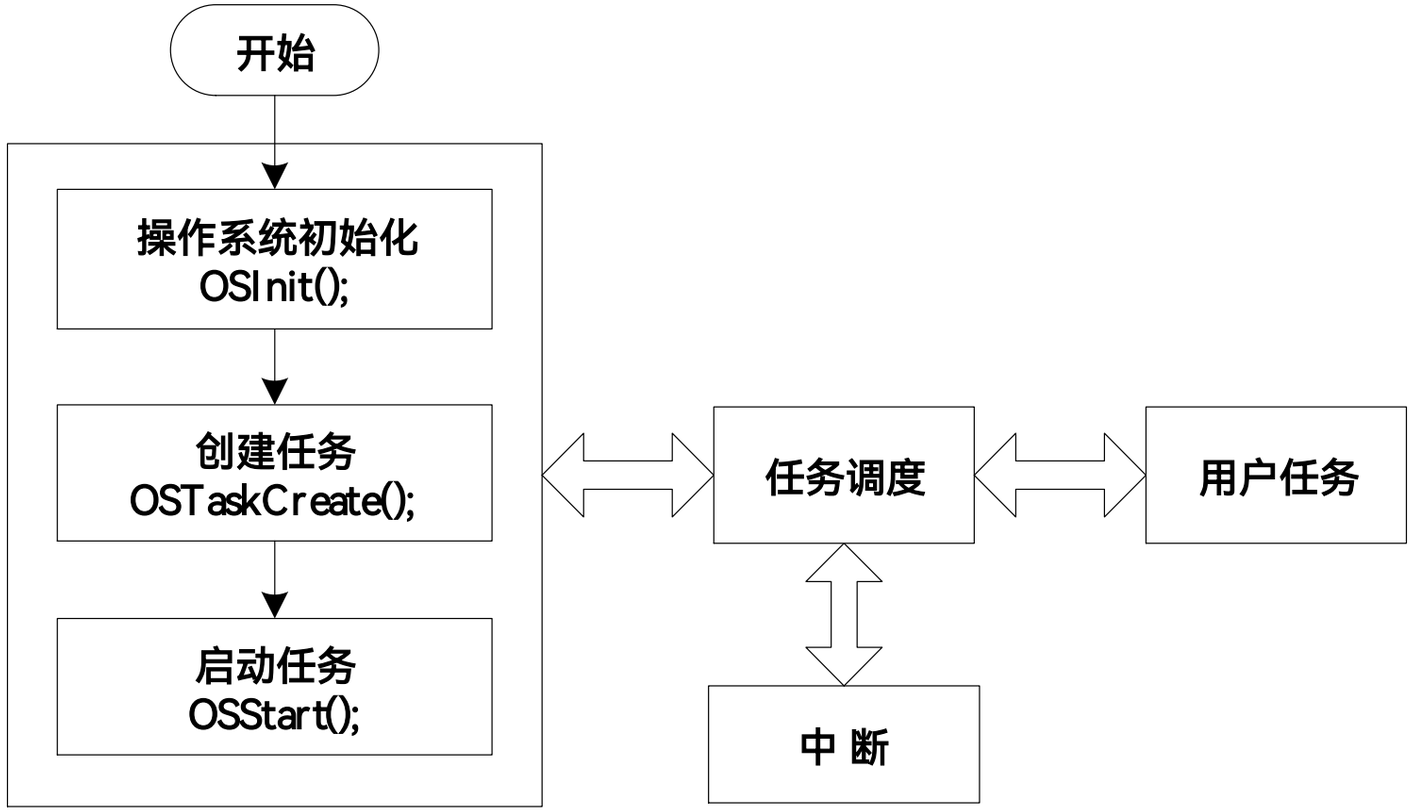
{
GPIO_WriteBit(GPIOB,GPIO_Pin_5,(BitAction)((1-

GPIO_ReadOutputDataBit(GPIOB,GPIO_Pin_5))));
        //LED灯状态翻转
     EXTI_ClearITPendingBit(EXTI_Line3);
```

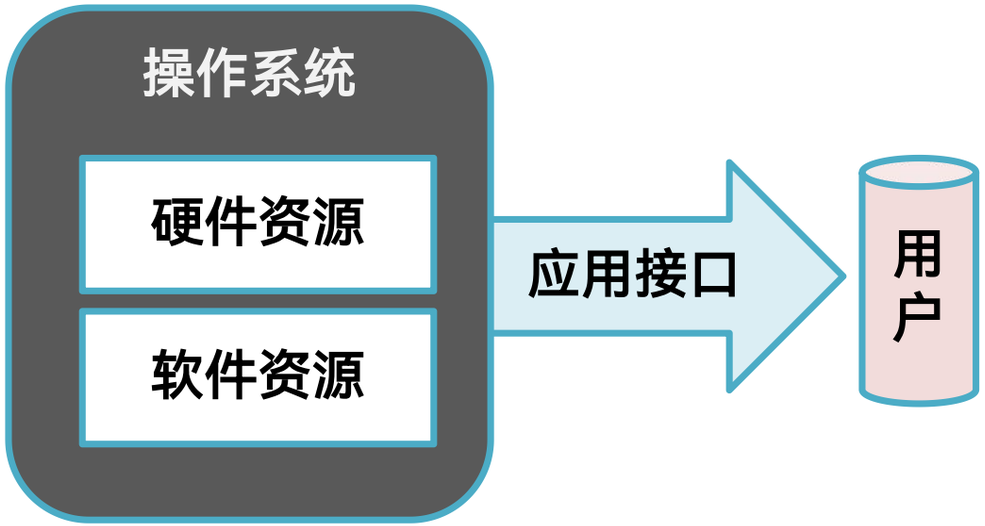
12.1.1 嵌入式系统的开发方式

多任务系统/基于嵌入式操作系统的方式

- 引入操作系统层
- 由操作系统对嵌入式平台的硬件资源进行统一管理和分配
- 这种方式实时性好，开发效率高。



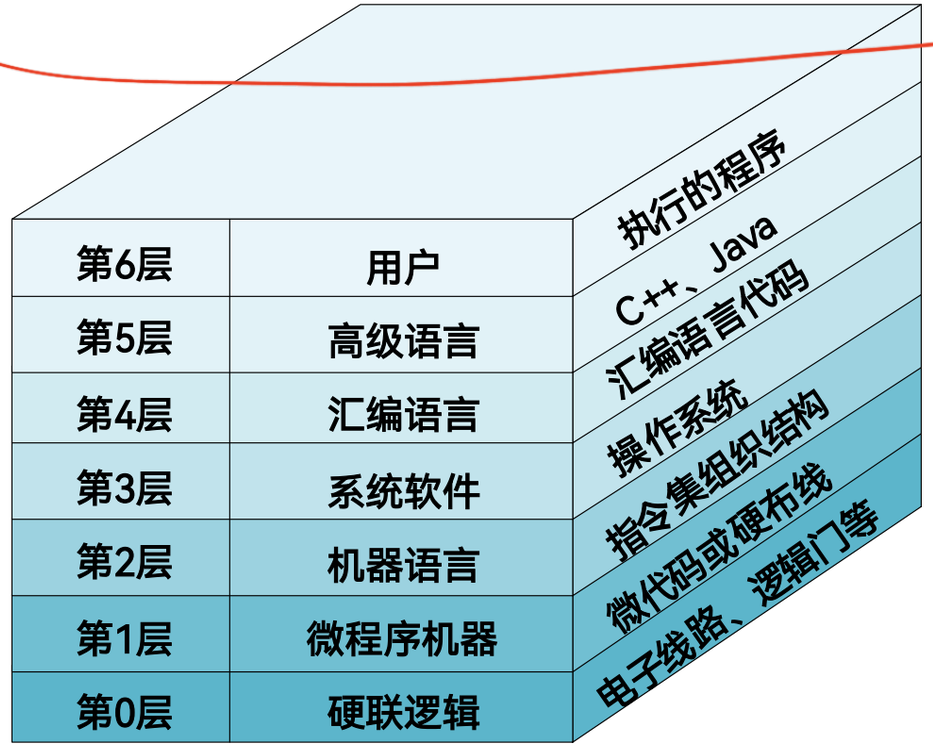
12.1.2 计算机系统的分层设计思想



- 把一个庞大而复杂的问题分解为若干个较小且易于处理的局部小问题；
- 局部小问题的功能部件的叠加完成整个系统功能。

分层设计思想应用在了计算机的各个领域，比如计算机系统的层次结构、计算机网络的分层结构等。

12.1.2 计算机系统的分层设计思想



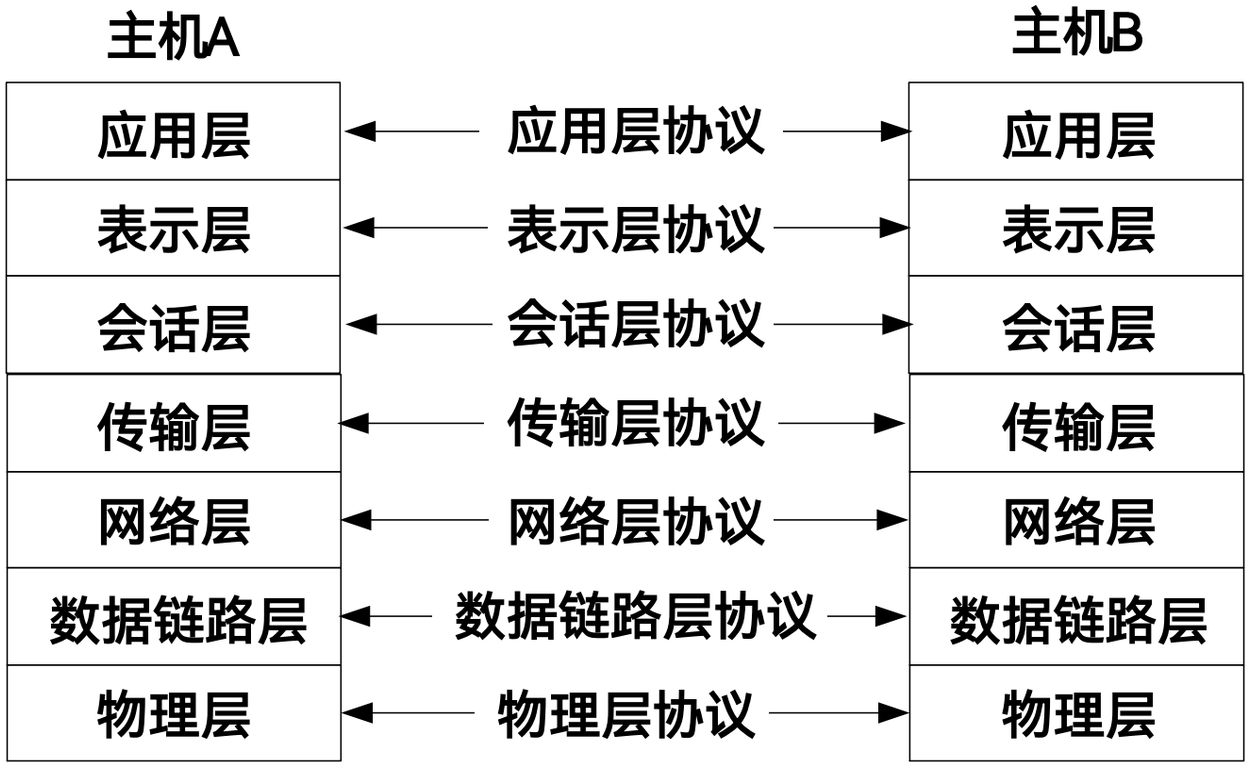
计算机系统层次结构中下层为上层提供服务，上层是下层的抽象和功能实现。

计算机系统的层次结构

- 第0层：最底层硬件，由逻辑门电路、电子线路等组成；
- 第1层：微程序级，由硬件直接执行；
- 第2层：传统的机器语言指令集；
- 第3层：系统软件，其核心是操作系统；
- 第4层：汇编语言层，由汇编编译程序支持和执行；
- 第5层：高级语言层，由各种高级语言编译程序支持和执行；
- 第6层：用户层，采用各种面向问题的应用语言。

12.1.2 计算机系统的分层设计思想

计算机网络的分层结构



提供应用程序间通信

处理数据格式、数据加密等

建立、维护和管理会话

建立端到端连接

寻址和路由选择

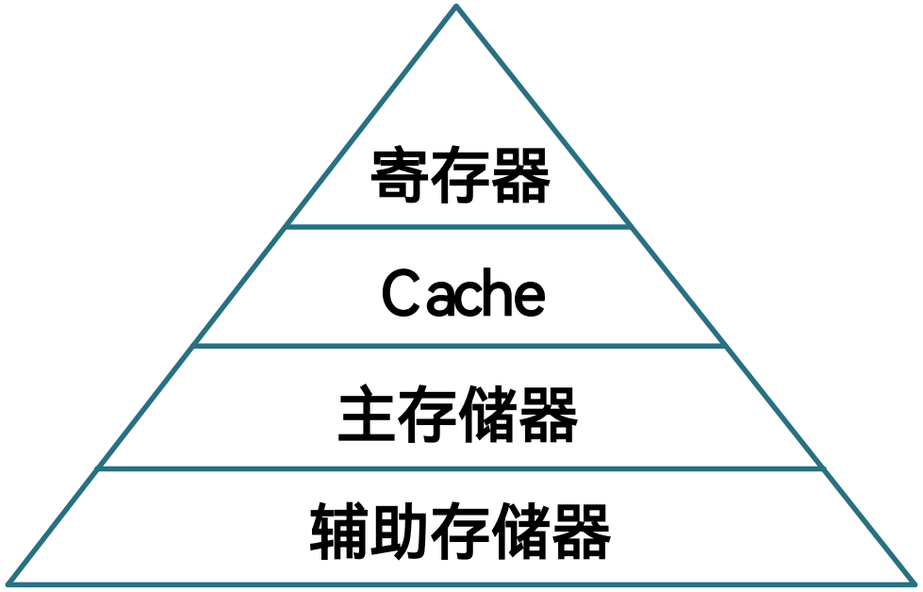
介质访问、链路管理

比特流传输

OSI的7层网络模型

12.1.2 计算机系统的分层设计思想

存储系统的层次结构

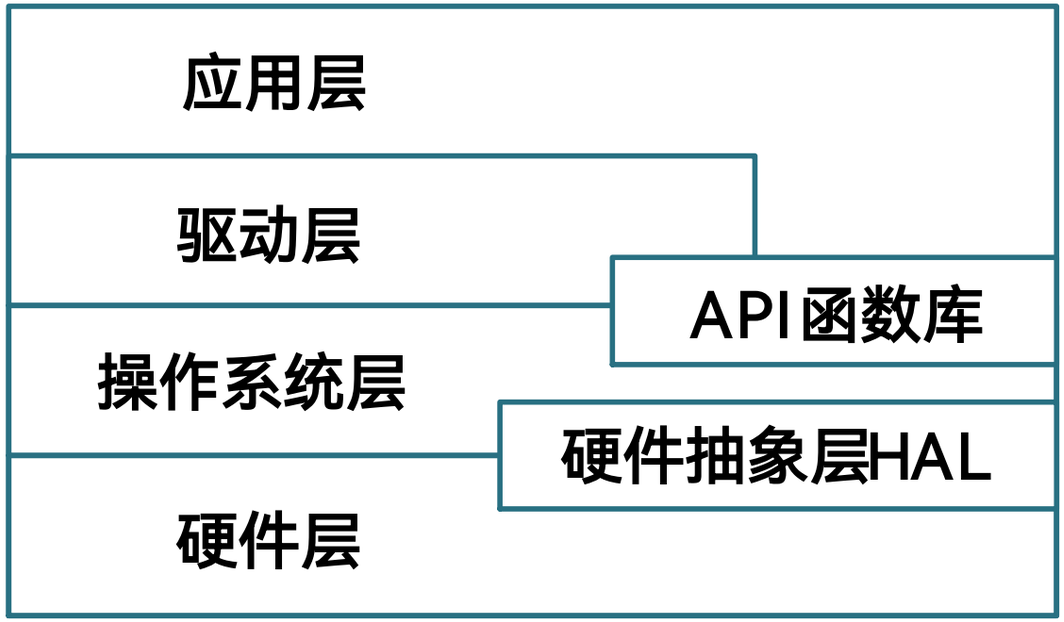


存储器层次结构图

通用计算机存储器结构分为四层：

- CPU寄存器：在CPU芯片内，寄存器中的数据直接在CPU内部参与运算，速度最快、容量最小；
- 高速缓冲存储器(Cache)：比主存速度更快、容量更小；
- 主存储器：存放将要参与运行的程序和数据；
- 辅助存储器：容量大、价位低，但速度慢，用来存放暂时未用到的程序和数据。

12.1.2 计算机系统的分层设计思想



嵌入式系统的层次结构

嵌入式系统层次结构

嵌入式系统结构分为4层

- 硬件层

操作系统和硬件之间的部分称为硬件抽象层(Hardware Abstraction Layer，HAL)

- 操作系统层
- 驱动层
- 应用层

分层的设计思想使得每一层只需知道下层抽象的接口，不需要了解其内部运行机制。

12.2 嵌入式操作系统的概念

1 12.2.1 计算机操作系统

2 12.2.2 嵌入式操作系统

3 12.2.3 嵌入式操作系统的分类

12.2.1 计算机操作系统

从用户的角度来看操作系统(Operation System，OS）的作用主要有：

作用1

- 对计算机底层硬件进行封装和功能的扩充。
- 操作系统对硬件进行了封装和隔离，只对用户提供一个简单有效的抽象接口，即用户与计算机之间的接口。

作用2

- 管理计算机资源；
- 程序运行时可使用的软、硬件环境统称为资源；
- 硬件：资源可以CPU、寄存器和I/O设备；
- 软件：资源可以是一个变量、一个结构或一个数组等。

12.2.1 计算机操作系统

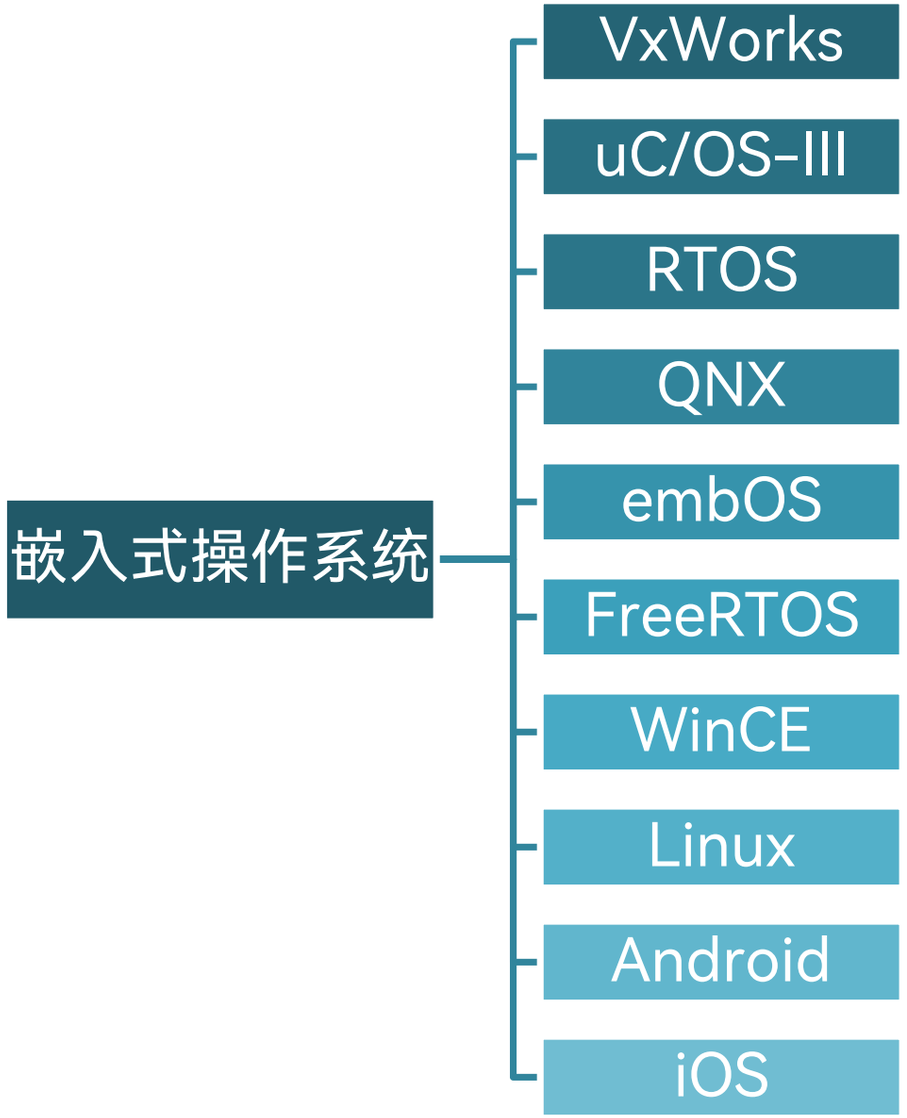
操作系统主要有五大管理功能

进程管理（CPU管理）	存储管理	设备管理	文件管理	作业管理（接口管理）
进程控制； 任务调度； 任务同步； 任务通信； 目标：公平、非阻塞、优先级。	包括：内存分配、内存保护、地址映射和内存扩充。 嵌入式操作系统：实模式和保护模式。	进行缓冲管理、设备分配和处理等管理，目的是屏蔽不同设备的差异性	为用户提供文件存储空间的存储管理、目录管理、文件的读/写管理以及文件的共享与保护等功能	用户通过操作系统提供的接口来访问计算机的硬件资源。主要有命令接口、程序接口和图形用户接口GUI等

12.2.2 嵌入式操作系统

嵌入式操作系统（Embedded Operating System，EOS）是指用于嵌入式系统的操作系统。

一般会有一个内核（Kernel），内核是指系统的一个组件，包含了操作系统的最基本、最主要功能，比如任务管理、存储管理、I/O设备管理等。

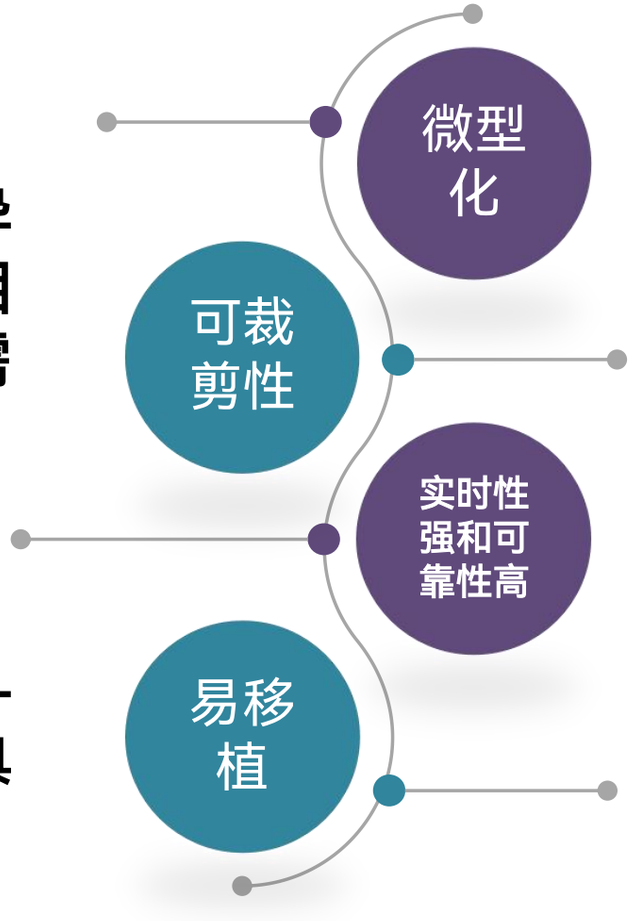


12.2.2 嵌入式操作系统

相比PC机操作系统，嵌入式操作系统具有以下的特点：

嵌入式系统应用环境差异很大，功能要求也各不相同，所以能够根据实际需求删减不需要的功能。

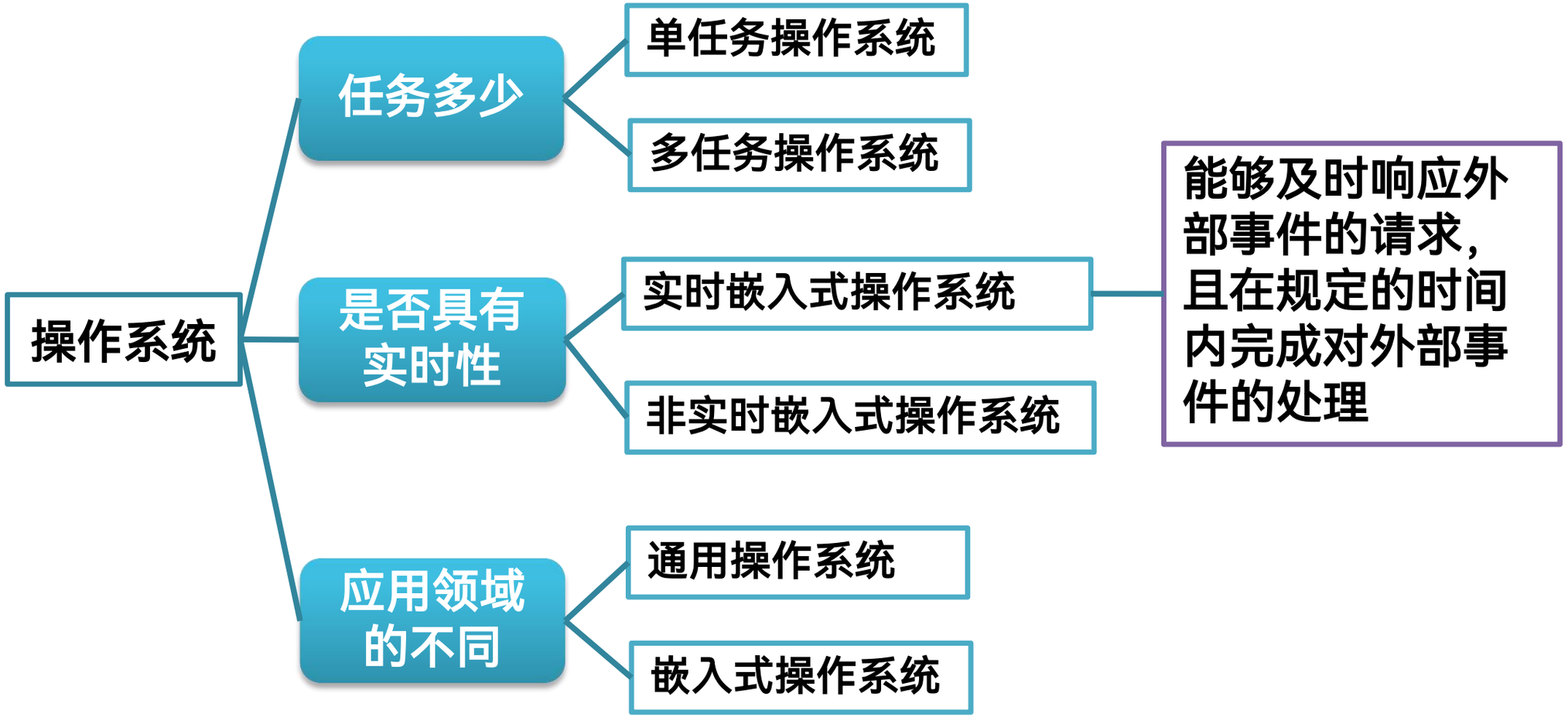
为满足多种硬件平台的统一性，要求嵌入式操作系统具有一定的可移植性。



嵌入式系统体积小，系统资源受限，要求嵌入式操作系统本身不能占用过多的存储空间和资源。

对实时性要求较高，且要求具备一定的抗干扰能力，可靠性高。

12.2.3 嵌入式操作系统的分类



12.2.3 嵌入式操作系统的分类

uC/OS-III就是一个基于优先级的抢占式多任务实时操作系统。

多任务管理

- 围绕多任务进行的；
- 将要解决的系统功能问题分解成若干个任务。



时间片轮转思想

- 按一定的时钟节拍分时处理任务；
- 按时间片分时轮流执行任务。

优先级分配问题

考虑实时性问题，优先级高的任务可以打断低优先级任务的执行。

12.3 嵌入式实时操作系统uC/OS-III



12.3.1 uC/OS-III的任务

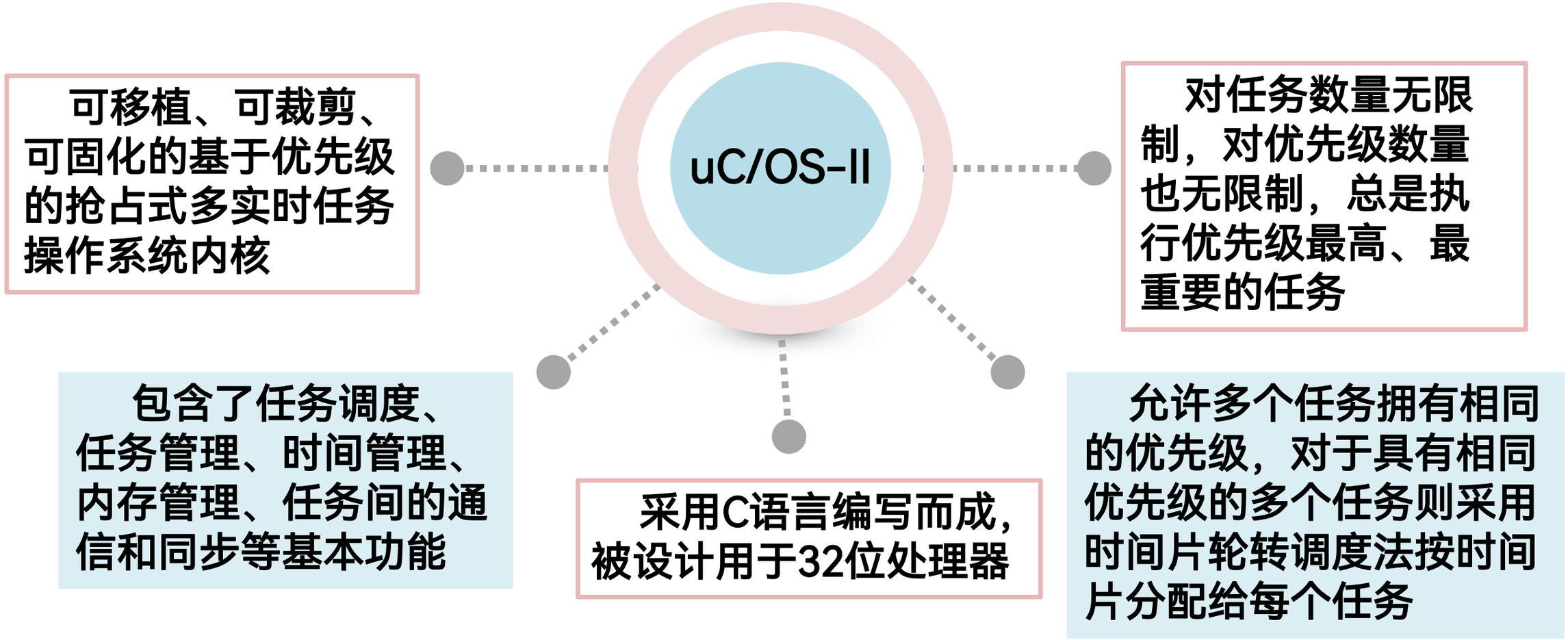


12.3.2 uC/OS-III时钟和中断

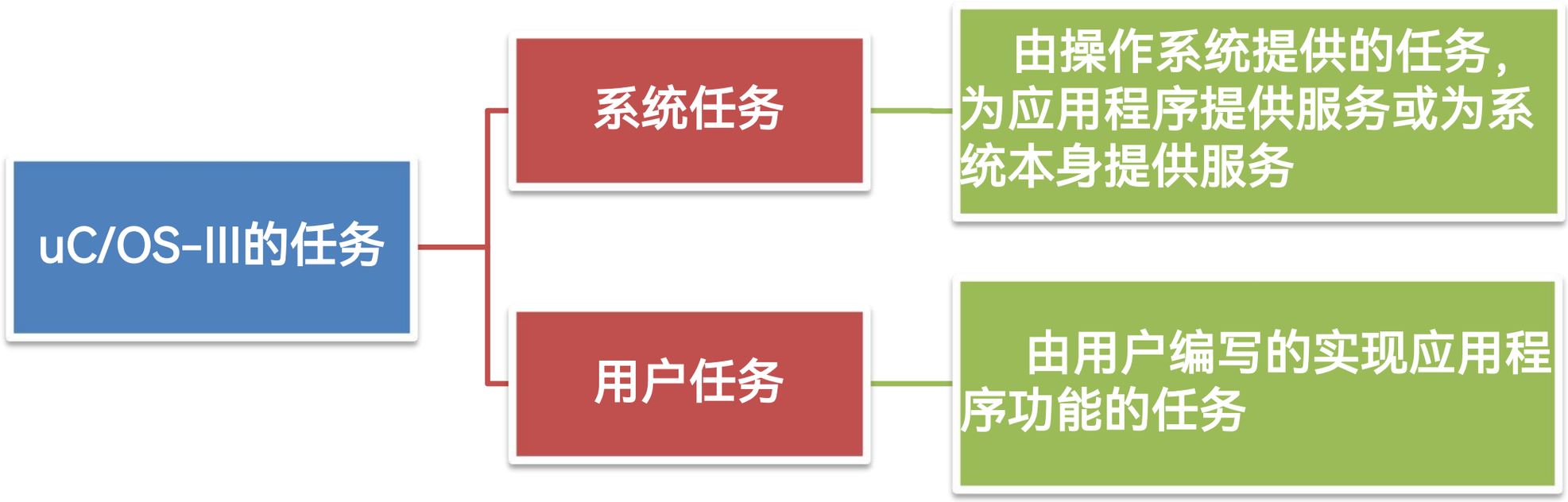


12.3.3 uC/OS-III任务间通信与同步

12.3 嵌入式实时操作系统uC/OS-III



12.3.1 uC/OS-III的任务

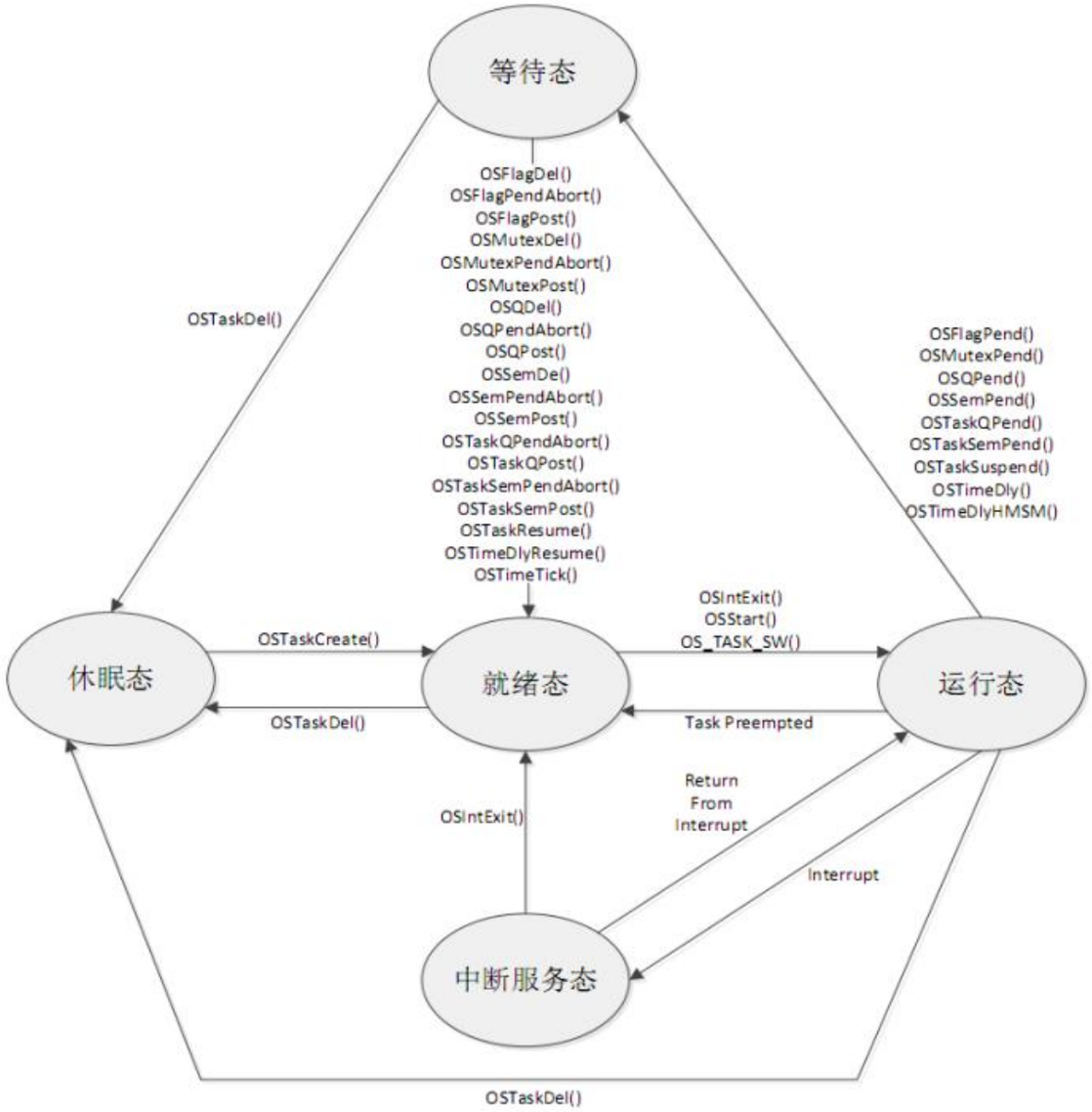


12.3.1 uC/OS-III的任务

uC/OS-III任务的状态

uC/OS-III中每个任务都是一个无限循环，每个任务都有五种状态：

- 休眠态
- 就绪态
- 运行态
- 挂起态
- 中断服务态



12.3.1 uC/OS-III的任务

uC/OS-III任务的状态

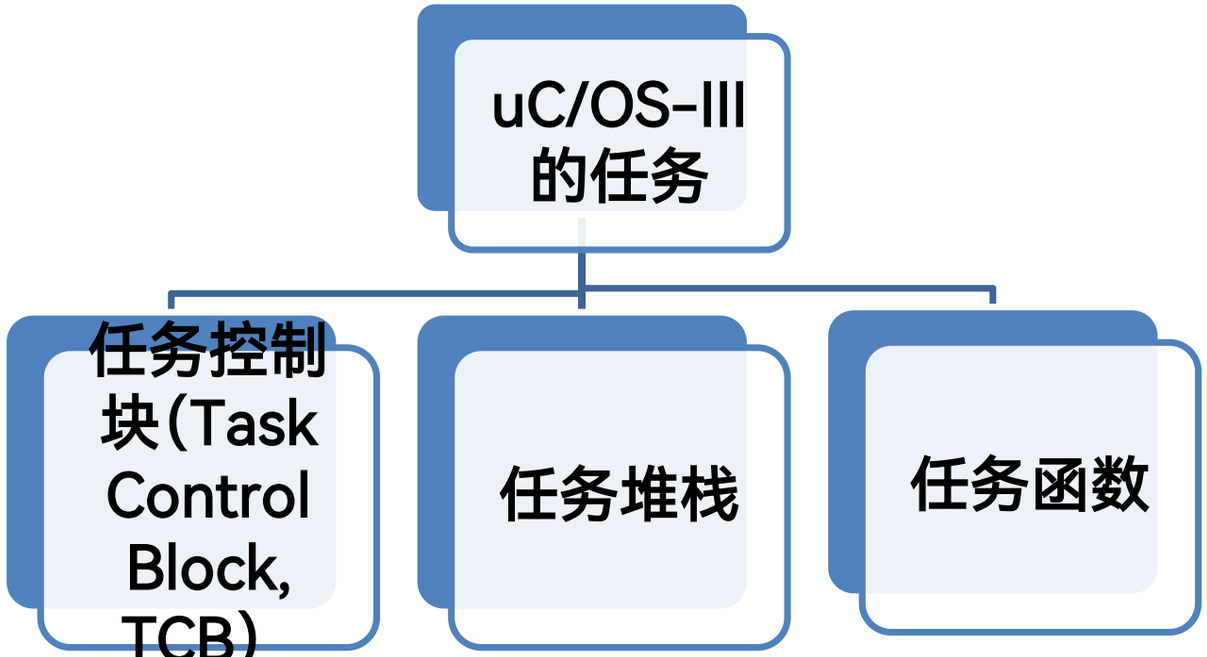
休眠态（Dormant）	就绪态（Ready）	运行态（Running）	等待态（Pending）	中断服务态（Interrupted）
任务只是以任务函数的方式存在，并未用OSTaskCreate()函数创建这个任务，任务不会被内核调度，也无法得到CPU的运行。	任务具备了运行的条件，并根据任务的优先级在系统的就绪表中排队，等待获得CPU的使用权。	正在运行的任务。任意时刻只能有一个任务处于运行态，调度器根据任务的优先级，使优先级最高的任务得到运行。	正在运行的任务需要等待某个事件或某个条件满足时才能继续运行，暂时让出CPU的使用权，进入等待事件状态。	正在执行的任务被中断打断，CPU转去执行中断服务程序，此时这个任务就会被挂起，进入中断服务状态。

12.3.1 uC/OS-III的任务

任务就是程序实体，uC/OS-III通过管理和调度这些任务来实现应用程序功能。

任务可看作是一个简单的程序，每个任务被赋予一定的优先级，有对应的CPU寄存器和堆栈空间，以保证任务运行所需的系统资源。

uC/OS-III任务的组成



12.3.1 uC/OS-III的任务

任务控制块

用来记录与任务相关的信息的数据结构，如任务堆栈指针、任务当前状态、任务优先级等。

每一个任务都有一个唯一的优先级别，通常也用任务的优先级作为这个任务的标识

记录了任务的全部静态和动态信息，每个任务都必须有各自的任务控制块。

系统需要为每个任务创建一个数据结构，称作任务控制块(Task Control Block, TCB)

多任务操作系统中，每个任务都有名字

任务控制块就相当于一个任务的身份证。

12.3.1 uC/OS-III的任务

任务控制块

任务控制块由用户创建，创建任务控制块的代码如下：

```
OS_TCB StartTaskTCB;
```

OS_TCB是一个结构体，定义在os.h文件中，结构体如下：

```
struct os_tcb{
CPU_STK   *StkPtr;
   //指向当前任务堆栈的栈顶
void   *ExtPtr; //指向用户可定义的数据区
CPU_STK   *StkLimitPtr;
   //可指向任务堆栈区中的某个位置
OS_TCB   *NextPtr;  //NextPtr和PrevPtr用于在任务就绪表建立OC_TCB双向链表
OS_TCB    *PrevPtr;
OS_TCB    *TickNextPtr; //TickNectPtr和TickPrevPtr可把正在延时或在指定时间内等待某个事件的任务的OC_TCB构成双向链表
OC_TCB      *TickPrevPtr;
OS_TICK_SPOKE *TickSpokePtr;
    //通过该指针可知道该任务在时钟节拍轮的哪个Spoke上
CPU_CHAR    *NamePtr;          //任务名
CPU_STK     *StkBasePtr        //任务堆栈基地址
OS_TASK_PTR *TaskEntryAddr;    //任务代码入口地址
void        *TaskEntryArg;     //传递给任务的参数
OS_PEND_DATA  *PendDataTblPtr;
   // 指向列表的指针，该列表包含挂起任务的对象信息
OS_STATE      PendOn;     //指示挂起的任务
OS_STATUS     PendStatus;  // 任务挂起的状态
OS_STATE      TaskState;  // 任务类型
OS_PRIO       Prio;       //任务优先级
CPU_STK_SIZE  StkSize;    //任务堆栈大小
OS_OPT        Opt;        // 可选参数
OS_OBJ_QTY    PendDataTblEntries;   //任务挂起对象数组的大小
CPU_TS        TS;         // 用于存储事件发生时的时间戳
OS_SEM_CTR    SemCtr;     // 计数型信号量的计数值
… …
}
```

12.3.1 uC/OS-III的任务 任务控制块

uC/OS-III把系统所有任务的控制块链接为两条链表来管理各个任务

空任务控制块链表

- 链表中的任务控制块都没有分配具体的任务。
- 该链表在系统调用初始化函数OSInit()函数时建立；
- 建立一个OS_TCB结构类型的数组OSTCBTbl[]实现；
- 每个元素都是一个任务控制块；
- 利用结构体中的两个指针OSTCBNext和OSTCBPre将任务控制块链接成一个链表。

任务控制块链表

- 链表中的任务控制块都已分配任务；
- 该链表在调用OSTaskCreate()函数时建立；
- 创建任务时，系统从空任务控制块链表的首部分配一个空任务控制块给该任务，并将它加入到任务控制块链表中。

12.3.1 uC/OS-III的任务

任务堆栈

作用：任务切换时用来保存任务的运行环境。

特点：每个任务都有自己的专用堆栈，彼此之间不能互用。

堆栈：内存中按照“后进先出（LIFO）”的原则组织的一块连续的存储空间。

任务堆栈的创建：

```
#define START_STK_SIZE  512
//堆栈大小
CPU_STK
START_TASK_STK[START_STK_SIZE];
 //定义一个数组来作为任务堆栈
```

定义了一个CPU_STK(任务堆栈）变量，空间大小为：512 * 4=2048字节。

12.3.1 uC/OS-III的任务

任务堆栈

使用OSTaskCreate()函数创建任务时，分配任务控制块的同时把所创建的任务堆栈传递给该任务。

任务堆栈基地址与堆栈的增长方向有关，堆栈的增长方向有两种：

- 向上增长：从低地址向高地址增长，任务堆栈基地址为&START_TASK_STK[0];
- 向下增长：从高地址向低地址增长，任务堆栈基地址为&START_TASK_STK[START_STK_SIZE-1]。

```
OSTaskCreate((OS_TCB     * )&StartTaskTCB,
//任务控制块
(CPU_CHAR   * )"start task",        //任务名字
(OS_TASK_PTR )start_task,           //任务函数
(void       * )0,  //传递给任务函数的参数
(OS_PRIO     )START_TASK_PRIO,     //任务优先级
(CPU_STK    * )&START_TASK_STK[0],
      //任务堆栈基地址
(CPU_STK_SIZE)START_STK_SIZE/10,
      //任务堆栈深度限位
(CPU_STK_SIZE)START_STK_SIZE,      //任务堆栈大小
(OS_MSG_QTY )0, //任务内部消息队列能够接收的最大消息数目,为0时禁止接收消息
(OS_TICK   )0,  //使能时间片轮转调度的时间片长度，为0时采用默认值
(void       * )0,    //用户补充的存储区
(OS_OPT     )OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR, //任务选项
(OS_ERR     * )&err);
                  //存放该函数错误时的返回值
```

12.3.1 uC/OS-III的任务

任务堆栈

使用OSTaskCreate()函数创建新任务时，会调用任务堆栈初始化函数OSTaskStkInit()

任务堆栈初始化函数OSTaskStkInit() 把任务初始数据存放到任务堆栈

```
CPU_STK  *OSTaskStkInit (OS_TASK_PTR p_task,
                         void           *p_arg,
                         CPU_STK        *p_stk_base,
                         CPU_STK        *p_stk_limit,
                         CPU_STK_SIZE    stk_size,
                         OS_OPT          opt)
```

```
{   CPU_STK  *p_stk;
    (void)opt;
    p_stk = &p_stk_base[stk_size];
    *--p_stk = (CPU_STK)0x01000000u;
    *--p_stk = (CPU_STK)p_task;
    *--p_stk = (CPU_STK)OS_TaskReturn;
    *--p_stk = (CPU_STK)0x12121212u;
    *--p_stk = (CPU_STK)0x03030303u;
    *--p_stk = (CPU_STK)0x02020202u;
    *--p_stk = (CPU_STK)p_stk_limit;
    *--p_stk = (CPU_STK)p_arg;
    *--p_stk = (CPU_STK)0x11111111u;
    *--p_stk = (CPU_STK)0x10101010u;
    *--p_stk = (CPU_STK)0x09090909u;
    *--p_stk = (CPU_STK)0x08080808u;
    *--p_stk = (CPU_STK)0x07070707u;
    *--p_stk = (CPU_STK)0x06060606u;
    *--p_stk = (CPU_STK)0x05050505u;
    *--p_stk = (CPU_STK)0x04040404u;
    return (p_stk);
}
```

12.3.1 uC/OS-III的任务

任务函数

由用户编写的任务处理代码，是应用功能的具体实现。uC/OS-III应用程序由多个任务组成，任务的一般写法如下：

```
void XXX_Task(void *p_arg)
{
  while(1)
  {
    …… //任务处理过程
  }
}
```

- 任务本质上是一个带有void *指针参数、无返回值的无限循环函数；
- 参数是一个void *类型的，目的是可以传递不同类型的数据或函数。

12.3.1 uC/OS-III的任务

uC/OS-III任务优先级

嵌入式操作系统支持多任务。

每个任务都具有优先级，任务越重要，赋予的优先级应越高。

uC/OS-III对优先级的数量没有限制，任务优先级数由宏OS_CFG_PRIO_MAX来配置，uC/OS-III将最低优先级只分配给空闲任务；

任务就绪表根据任务优先级顺序存放已经就绪的任务。

12.3.1 uC/OS-III的任务

uC/OS-III任务优先级

uC/OS-III的任务就绪表由两部分组成：

优先级位映射表OSPrioTbl[]

- 用来记录哪个优先级下有任务就绪。
- 当某个任务处于就绪状态就会将优先级映射表OSPrioTbl[]中相应的位置“1”；
- 通过三个相关的函数根据优先级来实现就绪表的操作：

- OS_PrioGetHighest()函数用于找到就绪表中最高优先级的任务；
- OS_PrioInsert()函数用于将指定优先级任务在就绪表中的相应位置“1”；
- OS_PrioRemove()函数用于将指定优先级任务在就绪表中的相应位清零。

12.3.1 uC/OS-III的任务

uC/OS-III任务优先级

就绪任务列表
OSRdyList[]

- 记录同一个优先级下所有就绪的任务；
- 处于同一优先级的多个任务，利用OSRdyList[]来统一管理，每一个元素对应一个优先级；
- 同一优先级下的所有任务都通过链表来管理的，最先运行的永远是HeadPtr所指向的任务。

OSRdyList[]定义在os.h文件中，元素的类型为结构体类型os_rdy_list，代码如下：

```
struct  os_rdy_list {
    OS_TCB       *HeadPtr;
 //用OS_TCB构造链表，指向链表头
    OS_TCB       *TailPtr;
 //用OS_TCB构造链表，指向链表尾
    OS_OBJ_QTY   NbrEntries;
//此优先级下的任务数量
};
```

12.3.1 uC/OS-III的任务

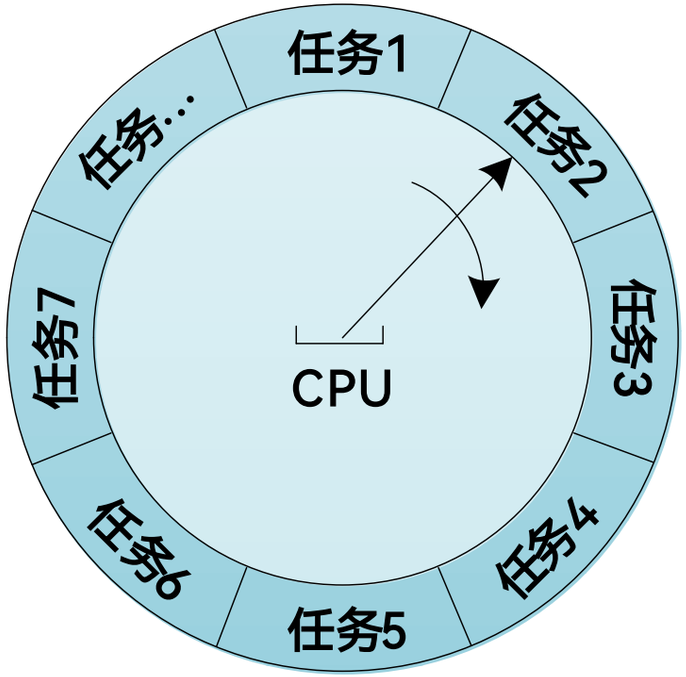
uC/OS-III任务调度和切换

任务调度协调任务对计算机系统资源的争夺使用，直接影响着系统的实时性能。

任务调度方式		
基于优先级的抢占式调度	不可抢占式调度	时间片轮转调度
内核总是将CPU分配给处于就绪态的优先级最高的任务运行，保证重要的突发事件及时得到处理。	任务一旦获得CPU就独占其运行，除非由于某种原因使它放弃CPU的使用权。	让优先级相同的处于就绪状态的任务按时间片轮流使用CPU，以防止同优先级的某一任务长时间独占CPU。

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换



时间片轮转调度示意图

- 大多数嵌入式操作系统采用基于优先级的**抢占式调度与时间片轮转调度相结合**的调度机制；
- uC/OS-III是一个抢占式多任务处理内核，同时又支持同优先级任务，对不同优先级的任务采用基于优先级的可抢占式调度法，对相同优先级的任务则采用时间片轮转调度法；
- uC/OS-III的任务调度由**任务调度器**完成，任务调度器有两种：任务级调度器和中断级调度器。

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换

- 任务之间的调度称为任务级调度，由函数OSSched()实现；
- 中断级的任务调度称为中断级调度，由函数OSIntExit()实现。

处于同一优先级的多个任务，uC/OS-III采用时间片轮转调度来轮流执行，要使用这个功能需要定义OS_CFG_SCHED_ROUND_ROBIN_EN为“1”

```
#define OS_CFG_SCHED_ROUND_ROBIN_EN
1u
```

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换

OS_SchedRoundRobin()函数用来执行时间片轮转调度，位于os_core.c文件，代码如下：

```
void  OS_SchedRoundRobin (OS_RDY_LIST  *p_rdy_list)
{    OS_TCB   *p_tcb;
     CPU_SR_ALLOC(); // 宏定义，定义变量cpu_sr，用来临时保存CPU的状态寄存器的值
    if (OSSchedRoundRobinEn != DEF_TRUE)
   {   // 确保时间片轮转调度允许
        return;    }
    CPU_CRITICAL_ENTER();    // 进入临界区
    p_tcb = p_rdy_list->HeadPtr;
      // 获取当前任务的任务控制块OS_TCB
    if (p_tcb == (OS_TCB *)0)
    {    // 确保任务不为空
        CPU_CRITICAL_EXIT();    // 退出临界区
        return;    }
```

```
if (p_tcb == &OSIdleTaskTCB) {       // 确保当前任务不是空闲任务
        CPU_CRITICAL_EXIT();         // 退出临界区
        return;
    }
    if (p_tcb->TimeQuantaCtr > (OS_TICK)0) {  // 递减时间片计数器
        p_tcb->TimeQuantaCtr--;
    }
    if (p_tcb->TimeQuantaCtr > (OS_TICK)0) {  // 查看任务的时间片是否使用完
        CPU_CRITICAL_EXIT();                  // 退出临界区
        return;
    }
```

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换

```
// 查看当前任务优先级下的就绪任务列表中的任务数是否大于2
    if (p_rdy_list->NbrEntries < (OS_OBJ_QTY)2)
   {
    CPU_CRITICAL_EXIT();     // 退出临界区
        return;
    }
    if (OSSchedLockNestingCtr > (OS_NESTING_CTR)0)
   {   // 查看任务调度器是否上锁
        CPU_CRITICAL_EXIT(); // 退出临界区
        return;
    }
// 把当前任务的任务控制块OS_TCB移动到该任务就绪表的尾部
    OS_RdyListMoveHeadToTail(p_rdy_list);
```

```
// 从任务就绪表头部获取新任务的任务控制块OS_TCB
 p_tcb = p_rdy_list->HeadPtr;
// 查看新任务控制块OS_TCB的时间片长度是否未设置
    if (p_tcb->TimeQuanta == (OS_TICK)0)
    {
        p_tcb->TimeQuantaCtr =
        OSSchedRoundRobinDfltTimeQuanta;
      // 时间片长度设置成默认值
    }
    else
    {   p_tcb->TimeQuantaCtr = p_tcb->TimeQuanta;
            // 载入新任务的时间长度
    }
    CPU_CRITICAL_EXIT(); // 退出临界区
}
```

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换

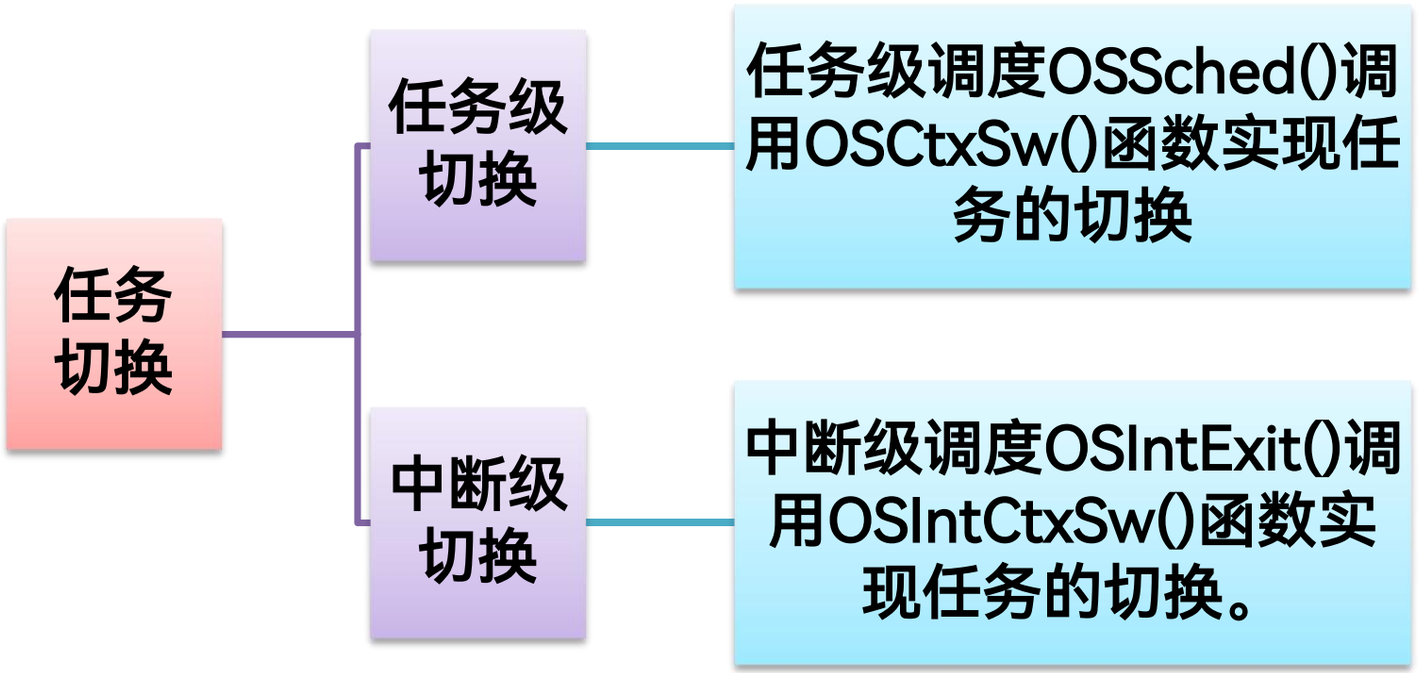
时间片轮转调度配置函数OSSchedRoundRobinCfg()位于os_core.c文件中，函数原型如下：

```
void  OSSchedRoundRobinCfg (CPU_BOOLEAN   en,    //打开/关闭时间片轮转调度机制
                            OS_TICK    dflt_time_quanta，//设置时间片长度，默认0为100ms。
                            OS_ERR      *p_err)    //调用本函数后返回的错误码
```

12.3.1 uC/OS-III的任务

uC/OS-III任务调度和切换

任务切换：当uC/OS-III需要切换到另外一个任务时，保存当前任务的现场(主要是CPU各寄存器的值）到当前任务的堆栈中，然后恢复新的现场(新任务的CPU各寄存器的值），并且执行新任务。



12.3.1 uC/OS-III的任务

任务管理决定了操作系统的实时性能,通常包含优先级设置、多任务调度机制和时间确定性等部分。

空闲任务 OS_IdleTask()	时钟节拍任务 OS_TickTask()	统计任务 OS_StatTask()	定时任务 OS_TmrTask()	中断服务管理任务 OS_IntQTask()
任务只是以任 自动创建的第一个任务，是必须创建的任务，优先级最低，当CPU中没有其它就绪任务运行时，空闲任务就会被运行。	实时操作系统需要时基周期或系统周期，OS_TickTask()任务是操作系统必须创建的系统任务，也称作时基任务。	可选任务，用来统计CPU使用率和各个任务的堆栈使用量，由宏OS_CFG_STAT_TASK_EN控制是否使用。	向用户提供定时服务，可选任务，由宏OS_CFG_TMR_EN控制是否使用此任务。	可选任务，由宏OS_CFG_ISR_POST_DEFERRED_EN控制是否使用此任务。

12.3.1 uC/OS-III的任务

uC/OS-III任务管理

uC/OS-III提供了很多与任务相关的函数，这些函数定义在os_task.c文件中

函数	说明
OSTaskCreate();	创建任务
OSTaskDel();	删除任务
OSTaskSuspend();	挂起任务
OSTaskResume();	恢复任务
OSTaskChangePrio();	修改任务优先级
OS_TaskInit();	任务初始化
OS_TaskInitTCB();	初始化堆栈的默认值
OSTaskSemPend();	等待接收任务信号量
OSTaskSemPendAbort();	终止等待任务信号
OSTaskQPend();	等待接收一个消息
OSTaskQPost();	发送信息给任务

12.3.2 uC/OS-III时钟和中断

uC/OS-III时钟

- 时钟节拍(操作系统的时基)实现时间的管理(延时、超时确认等）；
- STM32采用Cortex-M3内核提供的SysTick定时器产生系统时钟节拍，是一个周期为毫秒级的系统时钟；
- uC/OS-III允许用户通过配置宏的值来设定系统时钟节拍的频率，范围为10~1000Hz。

os_cfg_app.h

```
#ifndef OS_CFG_APP_H
#define OS_CFG_APP_H

/*
*********************************************************************************************************************************************************
*                                                                CONSTANTS
*********************************************************************************************************************************************************
*/

                                                                        /* ---------------------- MISCELLANEOUS ------------------ */
#define  OS_CFG_MSG_POOL_SIZE            100u                           /* Maximum number of messages                          */
#define  OS_CFG_ISR_STK_SIZE             128u                           /* Stack size of ISR stack (number of CPU_STK elements)*/
#define  OS_CFG_TASK_STK_LIMIT_PCT_EMPTY  10u                           /* Stack limit position in percentage to empty         */


                                                                        /* ----------------------- IDLE TASK --------------------- */
#define  OS_CFG_IDLE_TASK_STK_SIZE       128u                           /* Stack size (number of CPU_STK elements)             */


                                                                        /* ------------------ ISR HANDLER TASK ------------------ */
#define  OS_CFG_INT_Q_SIZE                10u                           /* Size of ISR handler task queue                      */
#define  OS_CFG_INT_Q_TASK_STK_SIZE      128u                           /* Stack size (number of CPU_STK elements)             */


                                                                        /* ------------------- STATISTIC TASK ------------------- */
#define  OS_CFG_STAT_TASK_PRIO            11u                           /* Priority                                            */
#define  OS_CFG_STAT_TASK_RATE_HZ         10u                           /* Rate of execution (10 Hz Typ.)                      */
#define  OS_CFG_STAT_TASK_STK_SIZE       128u                           /* Stack size (number of CPU_STK elements)             */


                                                                        /* ------------------------ TICKS ----------------------- */
#define  OS_CFG_TICK_RATE_HZ            1000u                           /* Tick rate in Hertz (10 to 1000 Hz)                  */
#define  OS_CFG_TICK_TASK_PRIO            10u                           /* Priority                                            */
#define  OS_CFG_TICK_TASK_STK_SIZE       128u                           /* Stack size (number of CPU_STK elements)             */
#define  OS_CFG_TICK_WHEEL_SIZE           17u                           /* Number of 'spokes' in tick  wheel; SHOULD be prime  */

```

12.3.2 uC/OS-III时钟和中断

初始化时钟节拍SysTick定时器操作在应用程序app.c文件中的起始任务 static void AppTaskStart (void *p_arg) 中实现

```
static  void  AppTaskStart (void *p_arg)
{
CPU_INT32U  cpu_clk_freq;
CPU_INT32U  cnts;

OS_ERR    err;
(void)p_arg;
BSP_Init();
CPU_Init();
cpu_clk_freq = BSP_CPU_ClkFreq();
 //获取CPU内核时钟频率，即SysTick时钟
//根据用户设定的时钟节拍频率计算SysTick定时器计数值
cnts = cpu_clk_freq / (CPU_INT32U)OSCfg_TickRate_Hz;
OS_CPU_SysTickInit(cnts);

//初始化SysTick，设置系统定时器周期为1ms
Mem_Init();
… …
}
```

12.3.2 uC/OS-III时钟和中断

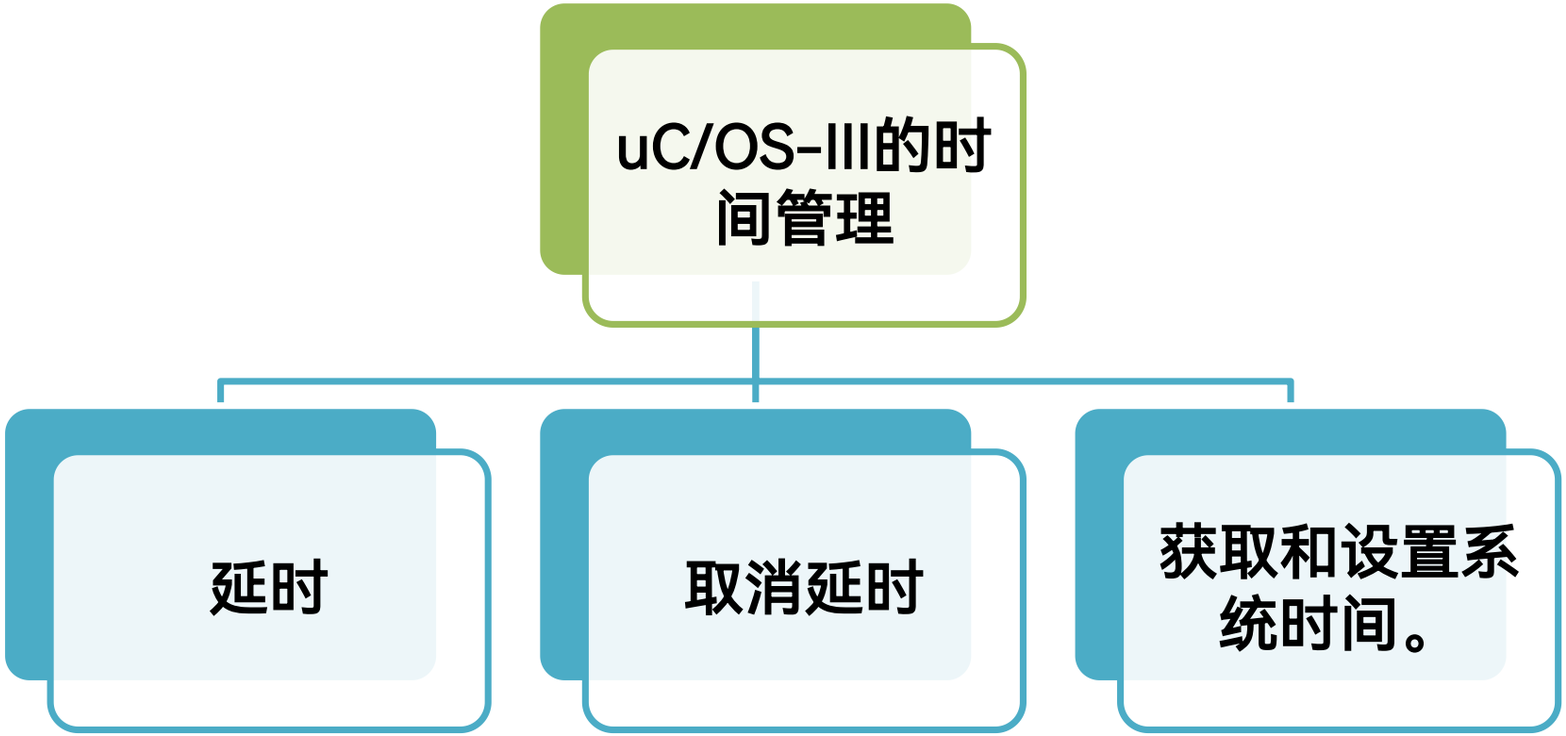
OS_CPU_SysTickInit()函数定义在os_cpu_c.c文件中，源码如下：

```
void  OS_CPU_SysTickInit (CPU_INT32U  cnts)
{
CPU_INT32U  prio; //定义一个优先级变量prio
//填写SysTick的重载计数值，SysTick 以该计数值为周期循环计数定时
CPU_REG_NVIC_ST_RELOAD = cnts - 1u;
prio = CPU_REG_NVIC_SHPRI3;
 //设置 SysTick 中断优先级
prio &= DEF_BIT_FIELD(24, 0);
//设置为默认的最高优先级0，在裸机例程中该优先级默认为最低
prio |=
DEF_BIT_MASK(OS_CPU_CFG_SYSTICK_PRIO,
24);
CPU_REG_NVIC_SHPRI3 = prio;
//使能SysTick的时钟源和启动计数器
CPU_REG_NVIC_ST_CTRL |=
CPU_REG_NVIC_ST_CTRL_CLKSOURCE |

CPU_REG_NVIC_ST_CTRL_ENABLE;
 //使能SysTick的定时中断
CPU_REG_NVIC_ST_CTRL |=
CPU_REG_NVIC_ST_CTRL_TICKINT;
}
```

12.3.2 uC/OS-III时钟和中断

uC/OS-III时间管理



12.3.2 uC/OS-III时钟和中断

uC/OS-III时间管理——延时函数

按节拍延时函数OSTimeDly()

```
void  OSTimeDly (OS_TICK  dly, OS_OPT  opt, OS_ERR
*p_err);
```

指向调用该函数后返回的错误码。

延时的时间节拍数，注意，节拍数必须在0~65535之间。

指定延时所使用的选项，共有四种：
OS_OPT_TIME_DLY(相对延时模式)
OS_OPT_TIME_TIMEOUT
OS_OPT_TIME_PERIODIC(周期性延时模式)
OS_OPT_TIME_MATCH(绝对定时模式）。

举例：如果时基频率设置为1000Hz，而任务每次执行延时大约2毫秒，则使用：
OSTimeDly(2，OS_OPT_TIME_DLY，&err);

12.3.2 uC/OS-III时钟和中断

uC/OS-III时间管理——延时函数

按时分秒毫秒延时函数OSTimeDlyHMSM()

```
void  OSTimeDlyHMSM (CPU_INT16U   hours,  //设置延时的小时数
                     CPU_INT16U   minutes,  //设置延时的分钟数
                     CPU_INT16U   seconds,  //设置延时的秒数
                     CPU_INT32U   milli,  //设置延时的毫秒数
                     OS_OPT       opt,  //选项，此函数只在相对延时模式下运
行
                     OS_ERR      *p_err)
```

举例：

```
OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HMSM_STRICT,&err);
```

12.3.2 uC/OS-III时钟和中断

uC/OS-III时间管理——取消延时

uC/OS-III允许处于延时的任务通过其他任务来取消延时从而使自己处于就绪状态，通过在其他任务中调用OSTimeDlyResume()函数来实现，OSTimeDlyResume()函数原型如下：

```
void  OSTimeDlyResume (
      OS_TCB  *p_tcb,
          //要被恢复延时的任务块
      OS_ERR  *p_err)
          //指向返回错误类型的指针
```

用户若想使用OSTimeDlyResume()函数，需将宏OS_CFG_TIME_DLY_RESUME_EN设为“1”。

```
#define
OS_CFG_TIME_DLY_RESUME_EN      1u
```

12.3.2 uC/OS-III时钟和中断

uC/OS-III时间管理——获取和设置系统时间

- uC/OS-III定义了一个CPU_INT32U类型(32位)的全局变量OSTickCtr来记录系统时钟节拍数
- 在调用OSInit()时被初始化为0，以后每发生1个时钟节拍，OSTickCtr加1，一直加到4294967295(232-1)，然后再从0开始计数，周而复始。

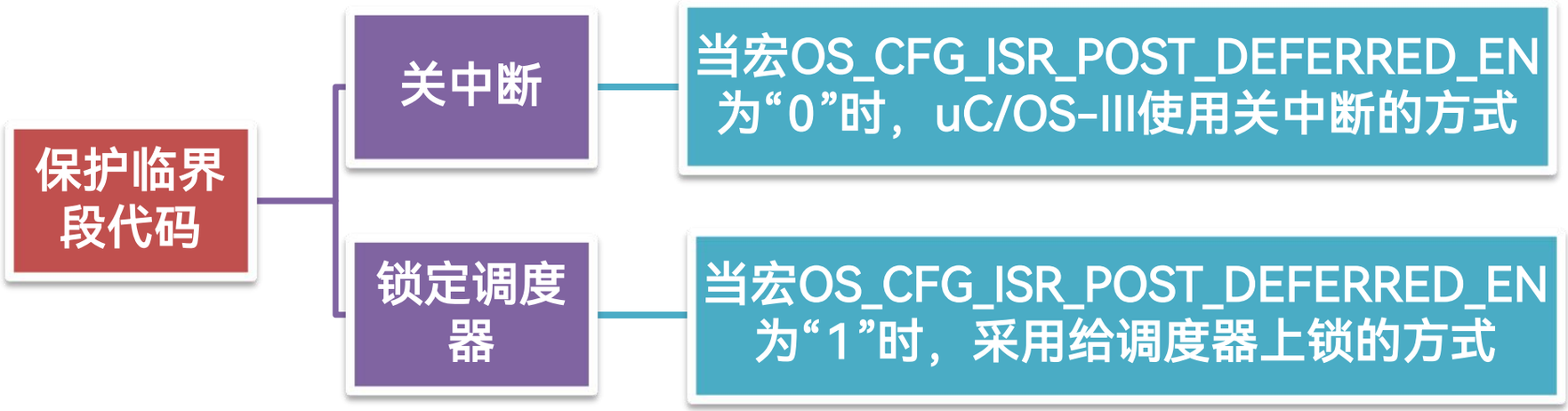
OSTimeGet()函数用于获取当前的时钟节拍计数值。

OSTimeSet()函数用于设置当前的时钟节拍计数值。

12.3.2 uC/OS-III时钟和中断

uC/OS-III临界段

- 临界段是指那些必须完整连续运行、不可被打断的代码段。



12.3.2 uC/OS-III时钟和中断

uC/OS-III临界段

- 临界段是指那些必须完整连续运行、不可被打断的代码段。

- uC/OS-III定义了进入临界段代码的宏OS_CRITICAL_ENTER()，而使用宏OS_CRITICAL_EXIT()和OS_CRITICAL_EXIT_NO_SCHED()退出临界区。
- OS_CRITICAL_ENTER()和OS_CRITICAL_EXIT()在使用时需成对使用，在这两条语句之间的就是临界段代码。

```
OS_CRITICAL_ENTER();
…… //临界段内的程序代码
OS_CRITICAL_EXIT();
```

12.3.2 uC/OS-III时钟和中断

uC/OS-III中断

- uC/OS-III支持中断，也允许中断嵌套，最多支持250级的中断嵌套
- uC/OS-III为了记录中断嵌套的层数，定义了一个全局变量OSIntNestingCtr。
- 在进入中断服务函数时要调用OSIntEnter()，将中断嵌套计数OSIntNestingCtr加1，OSIntEnter()定义在os_core.c文件中:

```
void  OSIntEnter (void)
{
    if (OSRunning !=
      OS_STATE_OS_RUNNING) {
        return;
    }
    if (OSIntNestingCtr >=
      (OS_NESTING_CTR)250u) {
        return;
    }
    OSIntNestingCtr++;
}
```

12.3.2 uC/OS-III时钟和中断

在退出中断服务函数时需要调用OSIntExit()函数，将中断嵌套计数OSIntNestingCtr减1，查看是否需要调度任务。

```
void PPP_IRQHandler(void)
{
      OSIntEnter();
      //中断服务程序
     OSIntExit();
}
```

uC/OS-III中断

SysTick时钟节拍的中断服务函数就是一个典型的uC/OS-III中断服务函数，其源码如下

```
void  OS_CPU_SysTickHandler (void)
{CPU_SR_ALLOC(); //分配保存中断状态的局部变量
CPU_CRITICAL_ENTER()和CPU_CRITICAL_EXIT()之间形成临界段，进入临界段
CPU_CRITICAL_ENTER();
OSIntNestingCtr++;
 //统计中断嵌套，进入中断时中断嵌套层数加1
 //退出临界段
CPU_CRITICAL_EXIT();
OSTimeTick();  //调用OSTimeTick()更新系统时钟节拍
OSIntExit();  //退出中断，中断嵌套层数减1
}
```

12.3.3 uC/OS-III任务间通信与同步

更多被用来实现任务间的同步以及任务和ISR之间的同步

当一个任务与多个任务发生同步时，推荐使用事件标志组

uC/OS-III提供了**信号量**、**事件标志组**等机制来实现任务之间的同步或者任务与中断服务程序(ISR）之间的通信。

uC/OS-III通过消息队列等机制来提供通信服务，用来在任务之间传送任意数量的数据。

12.3.3 uC/OS-III任务间通信与同步

信号量

信号量机制用于控制对共享资源的保护，当有多个任务同时访问共享资源时，设立一个标志，用于表示该共享资源的占用情况。

形式一：二值型信号量

取值只能为“0”或“1”，一次只允许一个任务访问共享资源；

形式二：多值型信号量

允许多个任务同时访问共享资源，初值为N，有任务请求时，该信号量的值就会减1，当有任务释放共享资源时，该信号量的值就会加1。

12.3.3 uC/OS-III任务间通信与同步

信号量

uC/OS-III中若要使用信号量机制，需先使能信号量，信号量的使能配置位于os_cfg.h文件

```
/* ---------------------- MUTUAL EXCLUSION SEMAPHORES ---------
---------- */
#define OS_CFG_MUTEX_EN    1u  //使能或禁用互斥型信号量
#define OS_CFG_MUTEX_DEL_EN  1u // 使能或禁用OSMutexDel() 函数
#define OS_CFG_MUTEX_PEND_ABORT_EN  1u //使能或禁用
OSMutexPendAbort() 函数
/* ---------------------------- SEMAPHORES ---------------------
------ */
#define OS_CFG_SEM_EN  1u  //使能或禁用多值信号量
#define OS_CFG_SEM_DEL_EN   1u   //使能或禁用OSSemDel()函数
#define OS_CFG_SEM_PEND_ABORT_EN  1u
    //使能或禁用OSSemPendAbort()函数
#define OS_CFG_SEM_SET_EN  1u   //使能或禁用OSSemSet()函数
```

12.3.3 uC/OS-III任务间通信与同步

信号量

uC/OS-III中在os_sem.c文件中定义了有关多值型信号量的API函数

函数名	说明
OSSemCreate()	建立一个多值型信号量
OSSemPost()	释放或发布一个多值型信号量
OSSemPend()	等待一个多值信号量，与OSSemPost()相对应
OSSemPendAbort()	取消或中止对一个多值信号量的等待
OSSemDel()	删除一个多值信号量
OSSemSet()	用于设置多值信号量的计数值

12.3.3 uC/OS-III任务间通信与同步

消息队列

uC/OS-III通过消息队列机制实现多个任务间的消息传递，使能配置位于os_cfg.h文件中。

```
/* ---------- MESSAGE QUEUES ----------
---------- */
#define OS_CFG_Q_EN    1u
    //使能或禁用消息队列
#define OS_CFG_Q_DEL_EN  1u
   // 使能或禁用OSQDel()函数
#define OS_CFG_Q_FLUSH_EN   1u
   //使能或禁用OSQFlush()函数
#define OS_CFG_Q_PEND_ABORT_EN
1u
   //使能或禁用OSQPendAbort()函数
```

uC/OS-III中消息队列的API函数

函数名	说明
OSQCreate()	建立一个消息队列
OSQPost()	用于向消息队列发布一条消息
OSQPend()	等待一个消息队列的消息，与OSQPost()相对应
OSQPendAbort()	取消或中止对一个消息队列的等待
OSQDel()	删除一个消息队列
OSQFlush()	用于清空一个消息队列

12.4 uC/OS-III移植与应用

- 所谓**移植**就是对一个实时的内核进行修改，使其能在其他的微处理器或者微控制器上运行。
- 用高级语言编写的源程序经过不同的编译器编译后，会产生不同的机器代码，运行于各类不同指令集的CPU上，这就是**跨平台**的概念。

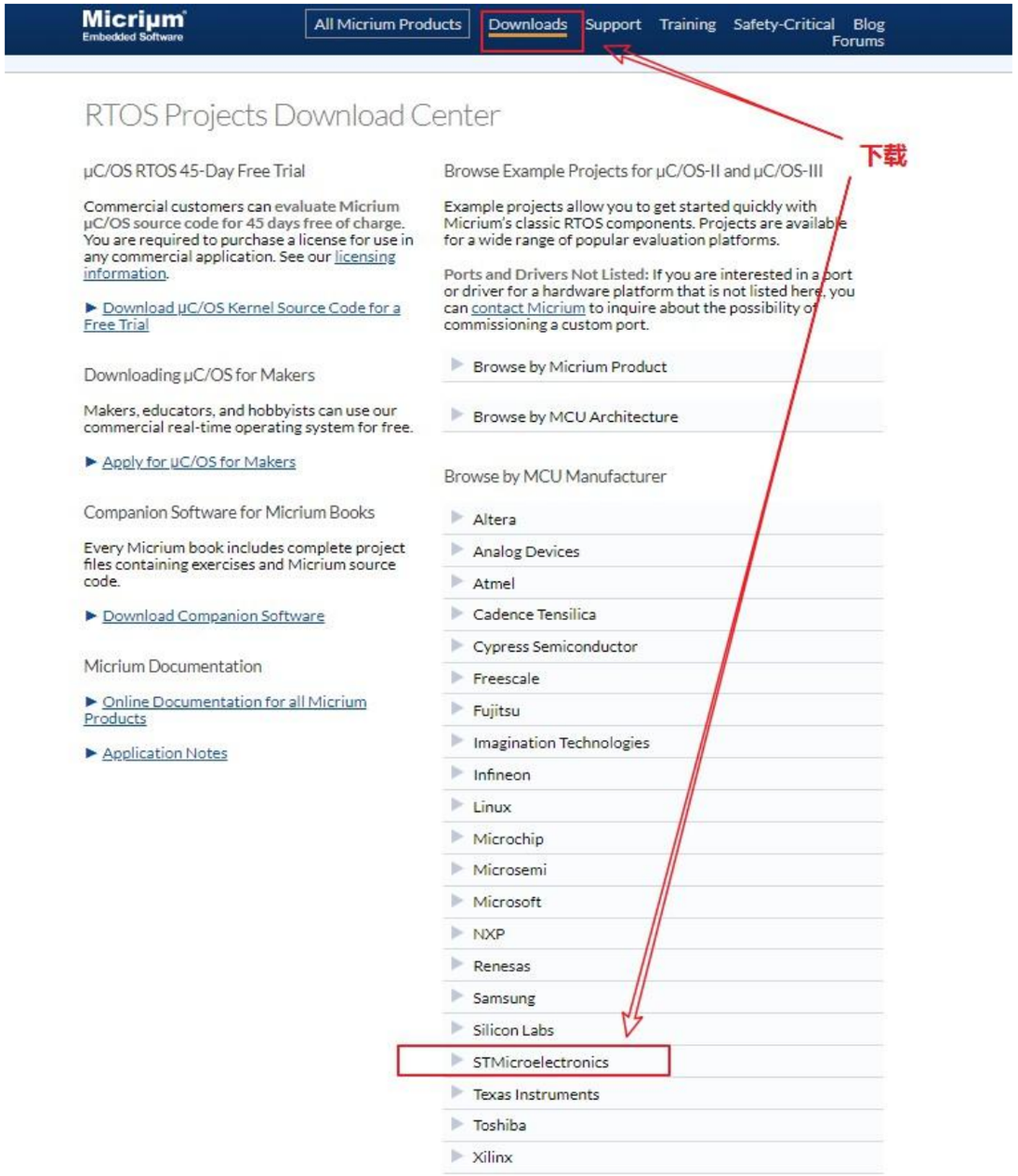
12.4.1 uC/OS-III源码下载

12.4.2 uC/OS-III移植

12.4.3 uC/OS-III多任务应用实例

12.4.1 uC/OS-III源码下载

登录Micrium公司官网，在“Downloads”中单击“STMicroelectronics”，列出了针对不同处理器平台的移植例程工程，如ST公司的STM32系列微控制器。



12.4.1 uC/OS-III源码下载

根据所用芯片的MCU厂商，选择相应的微处理器芯片型号



选择STM32F107平台的uC/OS-III源码

12.4.1 uC/OS-III源码下载

Projects

MCU	Micrium Product	Evaluation Board	Toolchain	Date
STMicroelectronics STM32F107	µC/TCP-IP µC/OS-III V3.04.04 µC/TCPIP V3.01.00 Library Project	Micrium uC-Eval-STM32F107	IAR (EWARM) V7.x	2014/08/15
STMicroelectronics STM32F107	µC/USB Device µC/OS-III V3.03.01 µC/USBD V4.01.01 Library Project	Micrium uC-Eval-STM32F107	IAR (EWARM) V6.x	2013/04/05
STMicroelectronics STM32F107	µC/TCP-IP µC/OS-III V3.03.01 µC/TCPIP V2.13.02 Library Project	Micrium uC-Eval-STM32F107	IAR (EWARM) V6.x	2013/04/05
STMicroelectronics STM32F107	µC/FS µC/FS V4.05.03 µC/OS-III V3.03.01 Library Project	Micrium uC-Eval-STM32F107	IAR (EWARM) V6.x	2013/04/02
STMicroelectronics STM32F107	µC/OS-III µC/OS-III V3.03.01	Micrium uC-Eval-STM32F107	Atollic TrueSTUDIO V3.x IAR (EWARM) V6.x Keil MDK V3.x	2013/02/08
STMicroelectronics STM32F107	µC/OS-II µC/OS-II V2.92.07	Micrium uC-Eval-STM32F107	Atollic TrueSTUDIO V3.x IAR (EWARM) V6.x Keil MDK V4.x	2013/02/08

下载uC/OS-III

下载uC/OS-II

针对Keil MDK开发平台的uC/OS-III下载

- 针对所使用的微处理器，选择合适的uC/OS-III版本；
- STM32F103ZET系列微控制器只有uC/OS-II版本，且只针对IAR开发平台，若使用Keil MDK开发平台，选择STM32F107下载基于Keil MDK开发平台的uC/OS-III

12.4.1 uC/OS-III源码下载



- 单击页面跳转到下载页面；
- 单击“Log in to Download”，需要先登录才能下载；
- 未注册的用户还需要先注册。

12.4.1 uC/OS-III源码下载

下载后解压，可以看到四个文件夹，分别为：

- EvalBoards
- uC-CPU
- uC-LIB
- uCOS-III，



uCOS-III文件结构

12.4.2 uC/OS-III移植

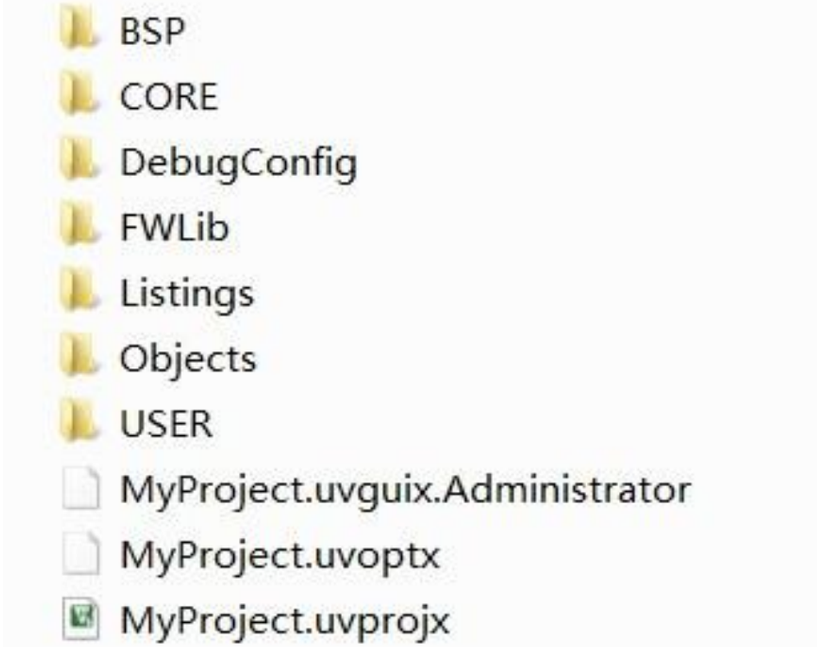
- 基于标准外设库v3.5.0的STM32工程模板上进行uC/OS-III移植；
- 需要一个基础的STM32工程模板，选择用GPIO中点亮一个LED灯的工程。



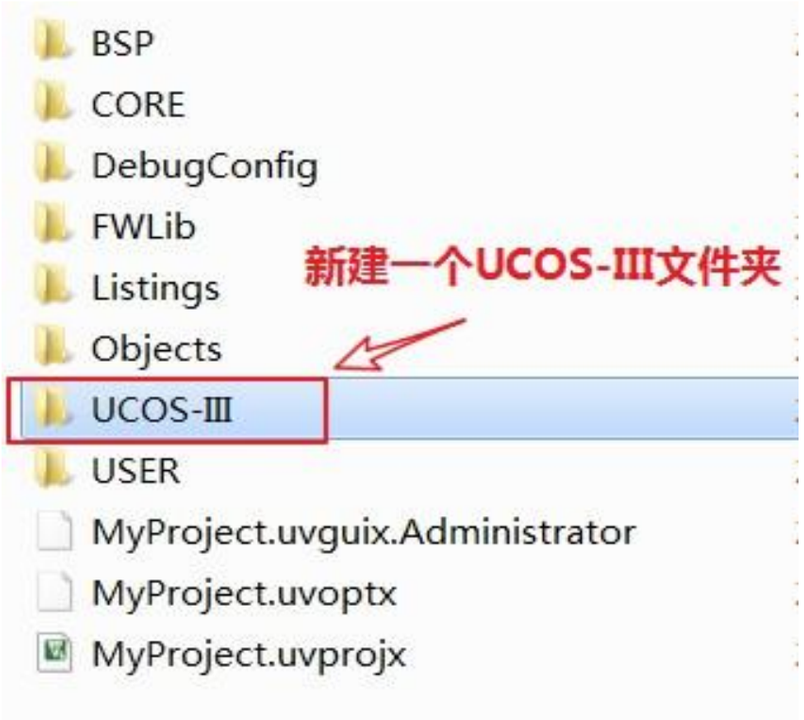
12.4.2 uC/OS-III移植

Step1：复制uC/OS-III源文件到SMT32工程中

1．将工程程序复制一份，并将其文件夹命名为“UCOS-III+LED”，工程目录如图



2．在该工程目录下新建一个文件夹，命名为“UCOS-III”



12.4.2 uC/OS-III移植

Step1：复制uC/OS-III源文件到SMT32工程中

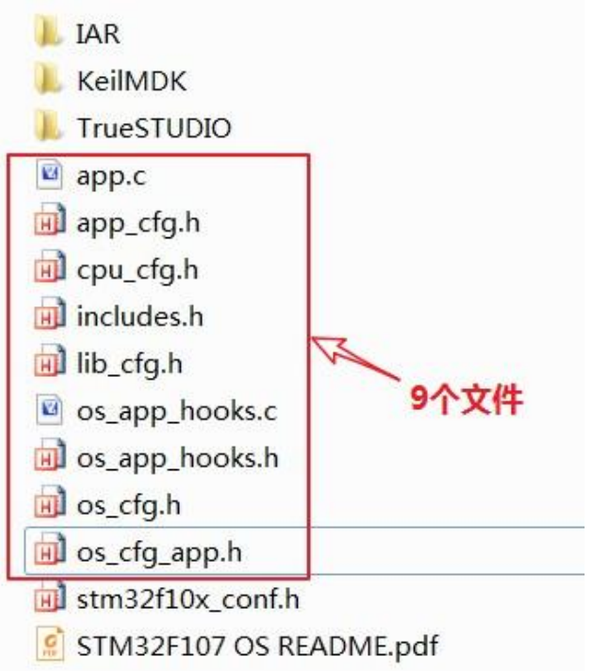
3．打开uC/OS-III源码文件，将源码中的三个文件夹uC-CPU、uC-LIB、uCOS-III复制到UCOS-III+LED工程目录下：
uC-CPU\ARM-Cortex-M3路径下的GNU和IAR两个文件夹删除；
uC-LIB\Ports\ARM-Cortex-M3路径下的GNU和IAR两个文件夹删除；
uCOS-III\Ports\ARM-Cortex-M3\Generic路径下的GNU和IAR两个文件夹删除；
新建一个UC-CONFIG文件夹，用于存放相关的配置文件。



12.4.2 uC/OS-III移植

Step1：复制uC/OS-III源文件到SMT32工程中

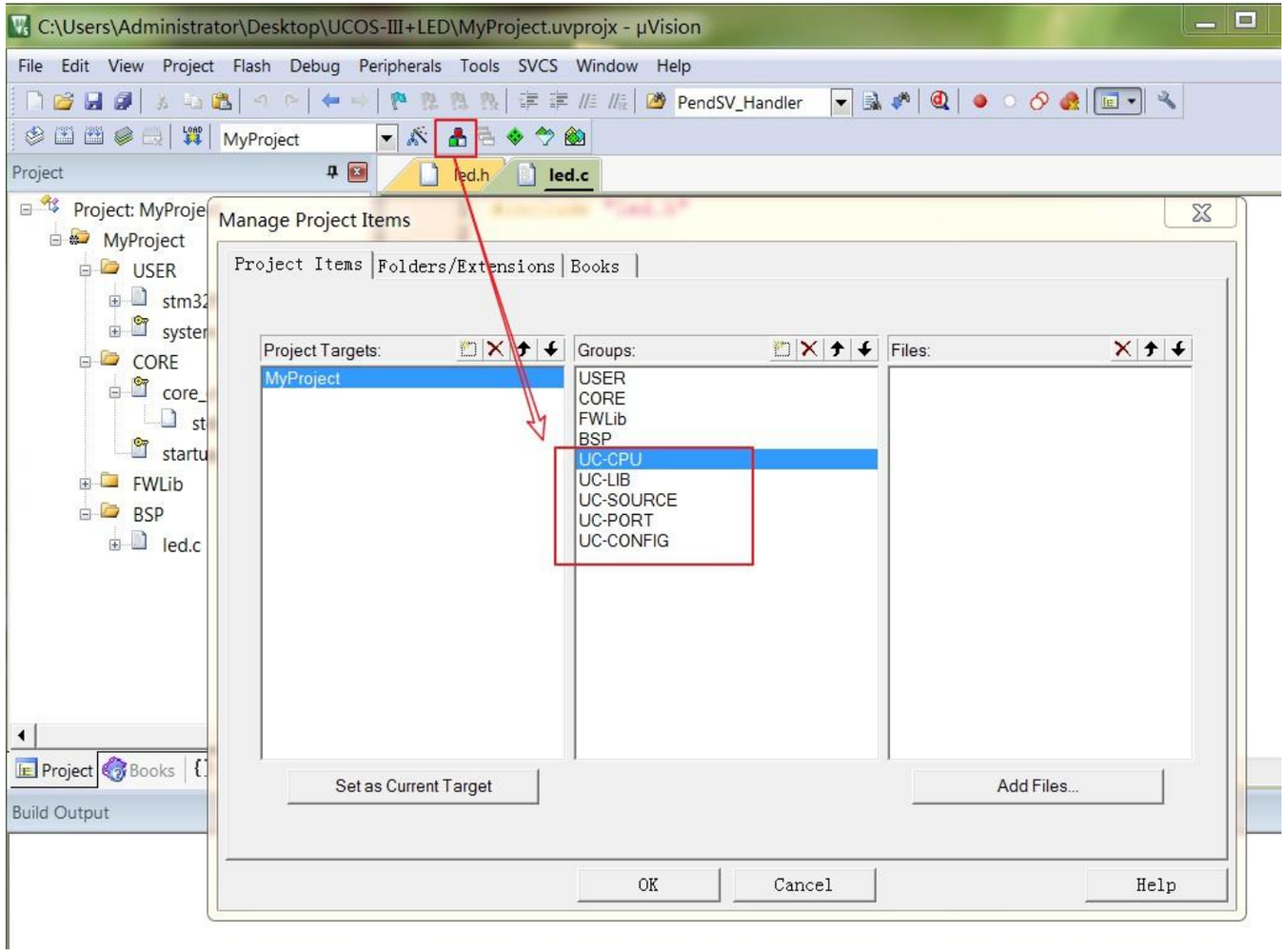
4．复制官方uC/OS-III源码中的相关文件到新建的UC-CONFIG文件夹，路径如下：\Micrium\Software\EvalBoards\Micrium\uC-Eval-STM32F107\uCOS-III，将其中的app.c、app_cfg.h、cpu_cfg.h、includes.h、lib_cfg.h、os_app_hooks.c、os_app_hooks.h、os_cfg.h、os_cfg_app.h等9个文件复制到UC-CONFIG文件夹下，将原工程



将Micrium\Software\EvalBoards\Micrium\uC-Eval-STM32F107\BSP文件夹下的bsp.c、bsp.h两个文件复制到UC-CONFIG文件夹中。

12.4.2 uC/OS-III移植

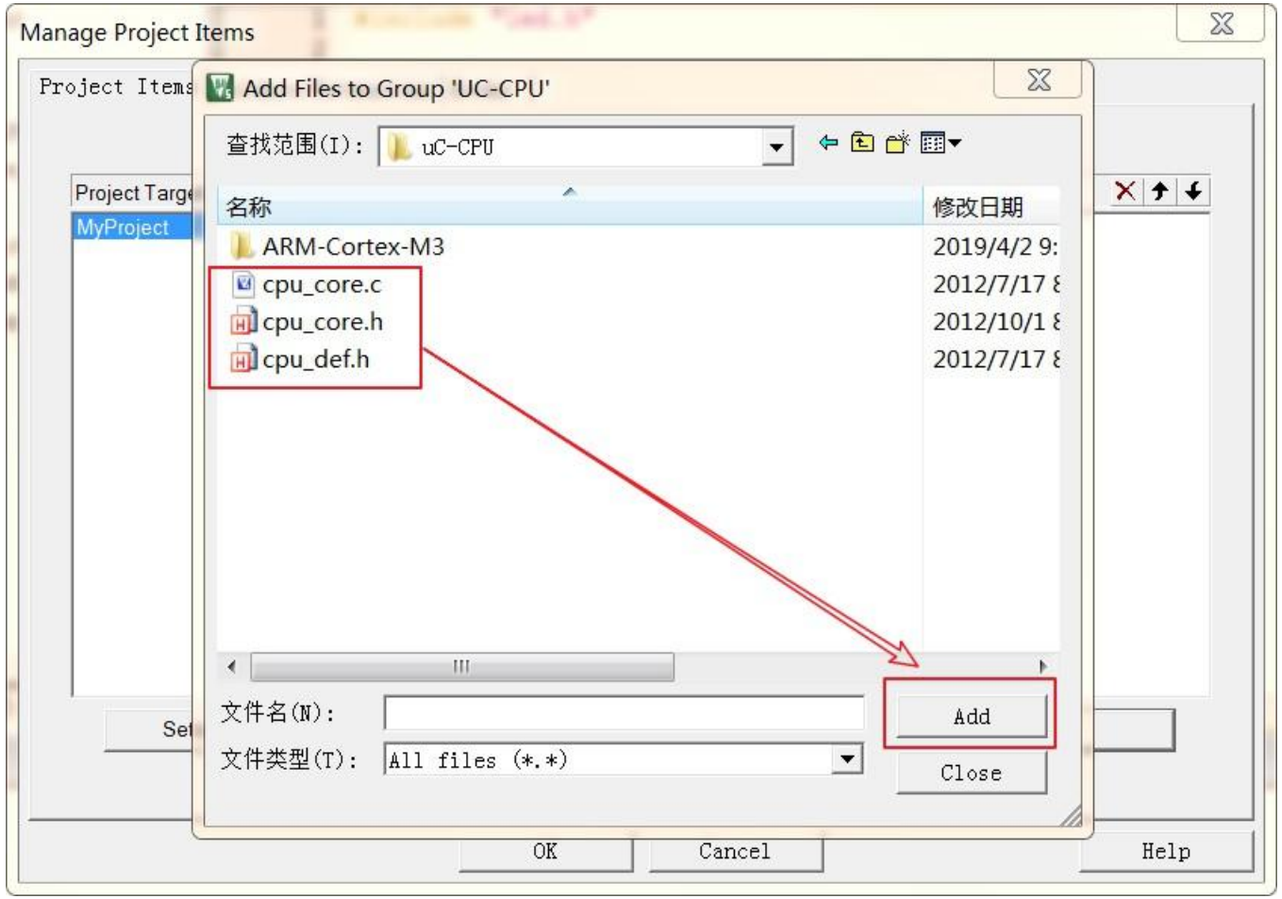
Step2：将uC/OS-III源文件添加进工程



6．打开工程，将uC/OS-III的源码添加到STM32工程中，单击Keil快捷菜单中的 按钮，在弹出的“Manage Project Items”的“Groups”一栏添加五个新的工程组：UC-CPU、UC-LIB、UC-SOURCE、UC-PORT、UC-CONFIG

12.4.2 uC/OS-III移植

Step2：将uC/OS-III源文件添加进工程

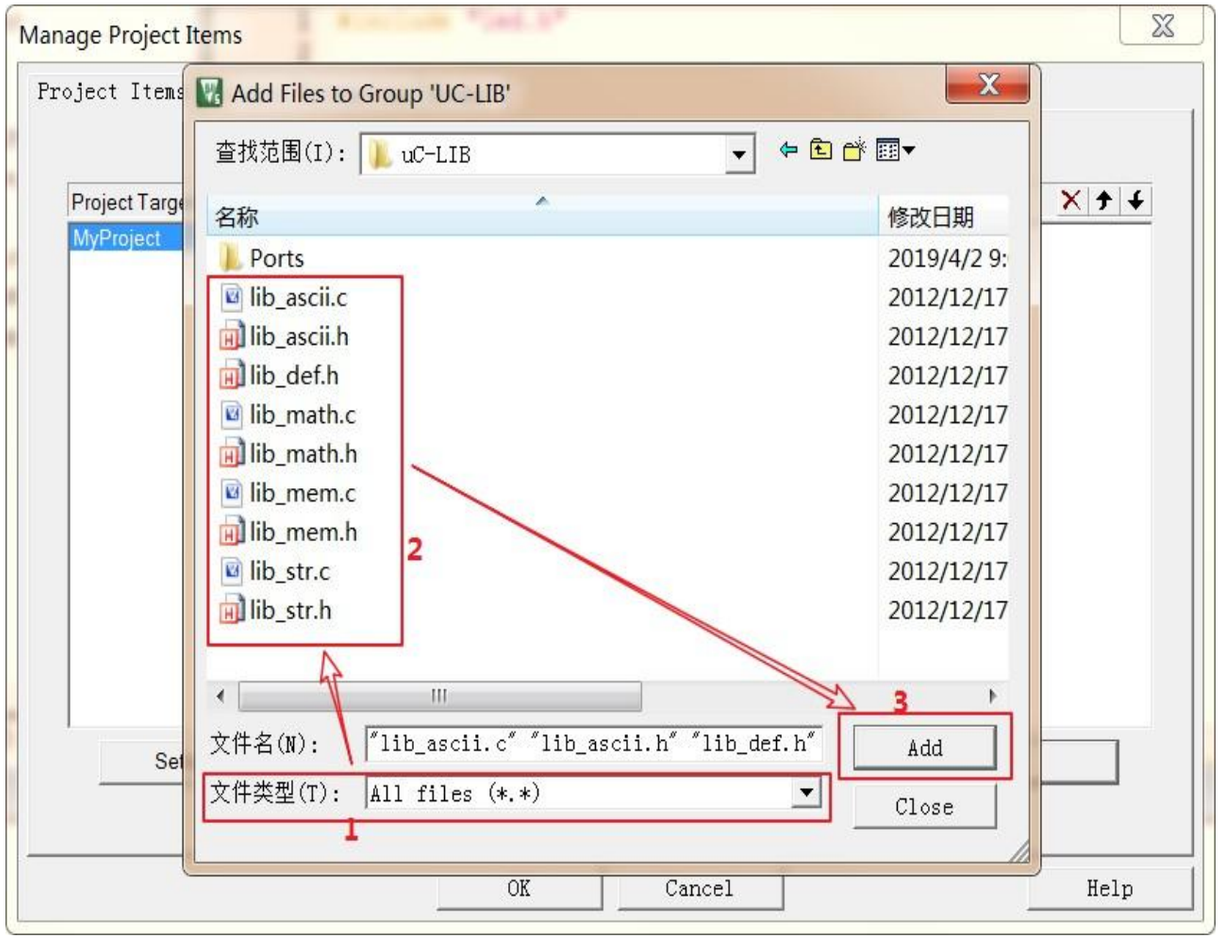


7．向UC-CPU工程组件中添加源文件。

- 选择UC-CPU组件，单击“Add Files…”按钮，跳转到UCOS-III/uC-CPU，选择文件类型为“All files(*.*)”，选中其中的三个文件，单击“Add”添加；
- 打开ARM-Cortex-M3\RealView文件夹，将cpu.h、cpu_a.asm、cpu_c.c添加进工程。

12.4.2 uC/OS-III移植

Step2：将uC/OS-III源文件添加进工程



8．向UC-LIB工程组件中添加源文件。

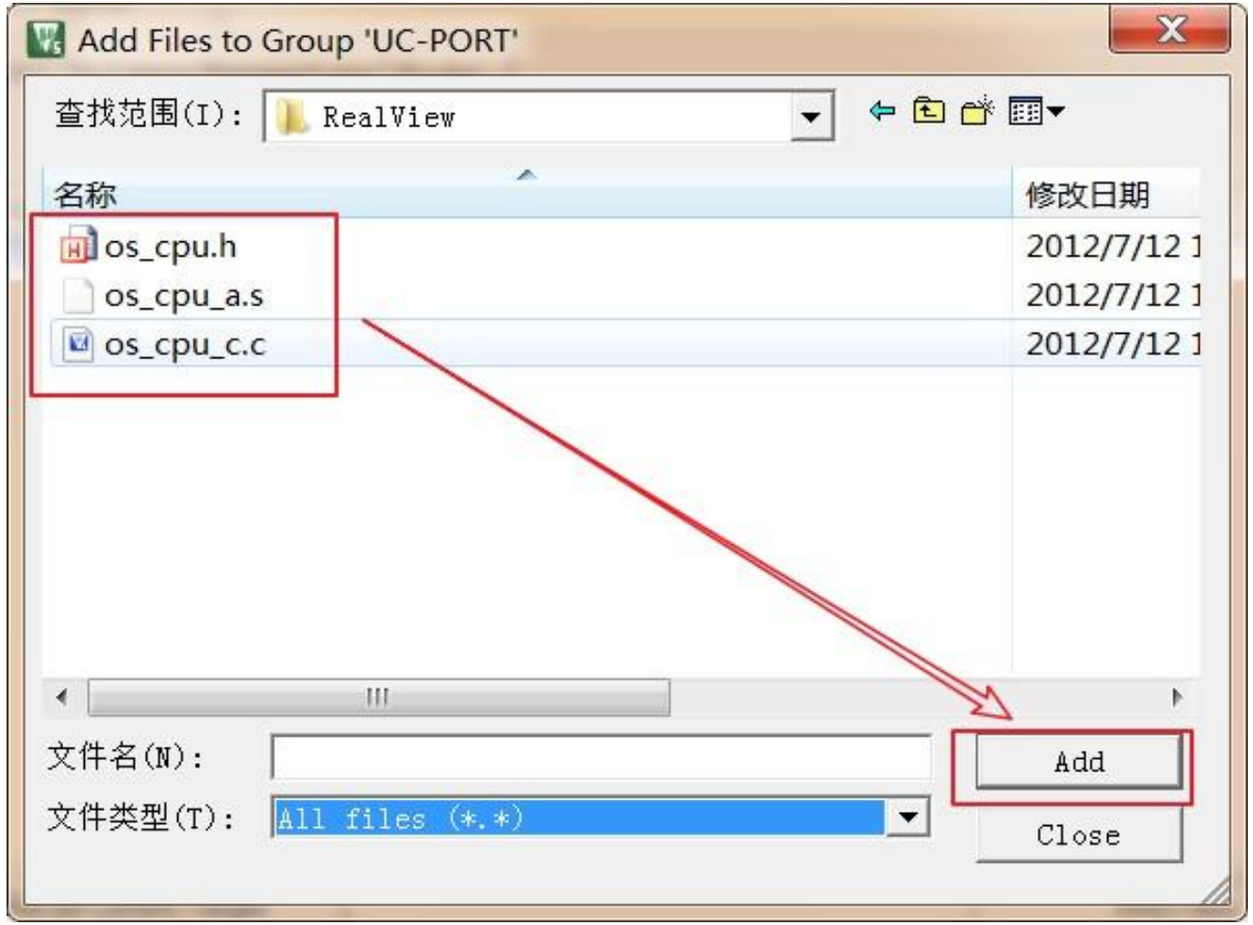
- 选择UC-LIB工程组件，单击“Add Files…”按钮，跳转到UCOS-III/uC-LIB，选择“All files(*.*)”文件类型，将其中的9个文件添加进UC-LIB工程组；
- 将Ports\ARM-Cortex-M3\RealView文件下的lib_mem_a.asm添加进工程组。

12.4.2 uC/OS-III移植

Step2：将uC/OS-III源文件添加进工程

10．向UC-PORT工程组件中添加源文件。

- 选择UC-PORT工程组件，单击“Add Files…”按钮，跳转到UCOS-III\uCOS-III\Ports\ARM-Cortex-M3\Generic\RealView，选择“All files(*.*)”文件类型，将其中的三个文件os_cpu.h、os_cpu_a.s、os_cpu_c.c添加进UC-PORT工程组件中。

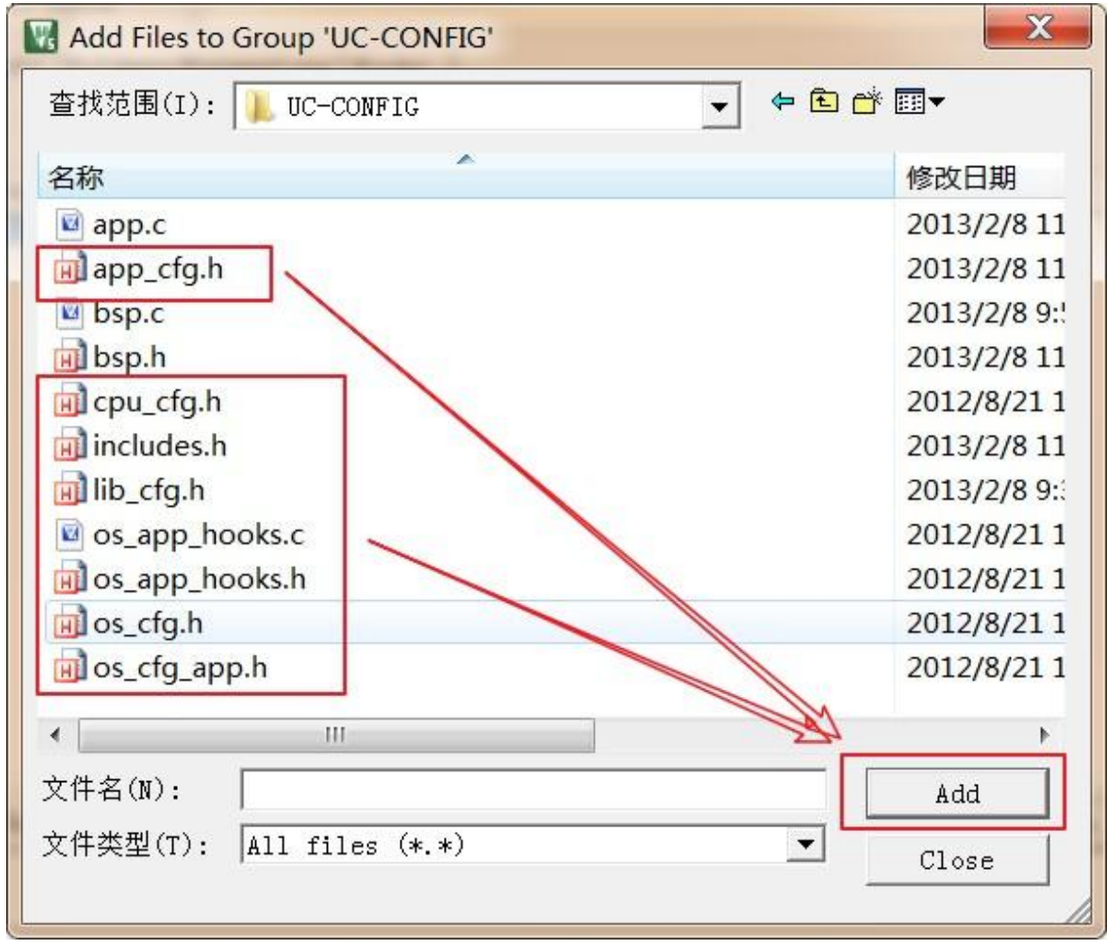


12.4.2 uC/OS-III移植

Step2：将uC/OS-III源文件添加进工程

11．向UC-CONFIG工程组添加源文件。

- 选择UC-CONFIG工程组件，单击“Add Files…”按钮，跳转到UCOS-III\UC-CONFIG，选择“All files(*.*)”文件类型，将其中的8个文件(除了app.c、bsp.c、bsp.h这三个文件）添加进UC-CONFIG工程组

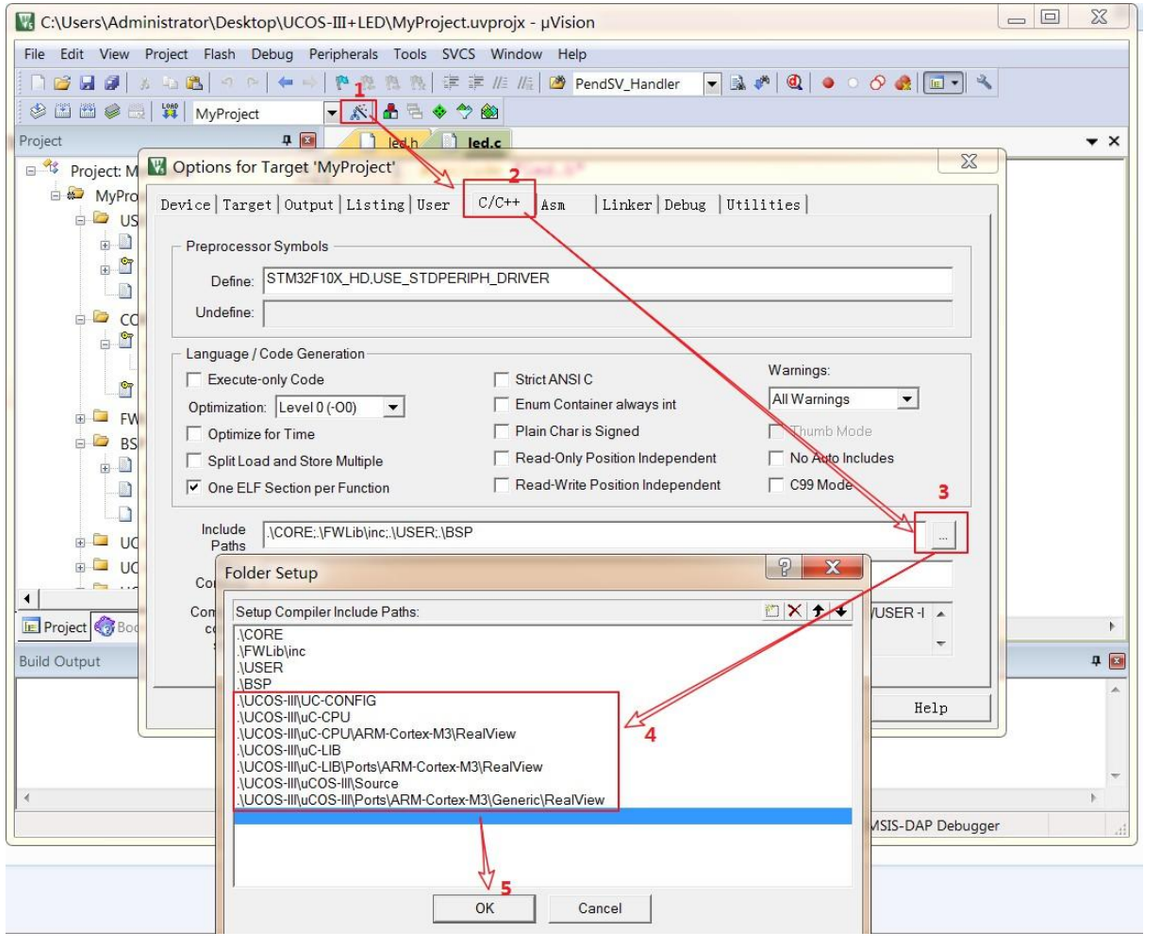


12.4.2 uC/OS-III移植

Step2：将uC/OS-III源文件添加进工程

12．添加app.c文件到USER工程组件，添加bsp.c、bsp.h文件到BSP工程组件。

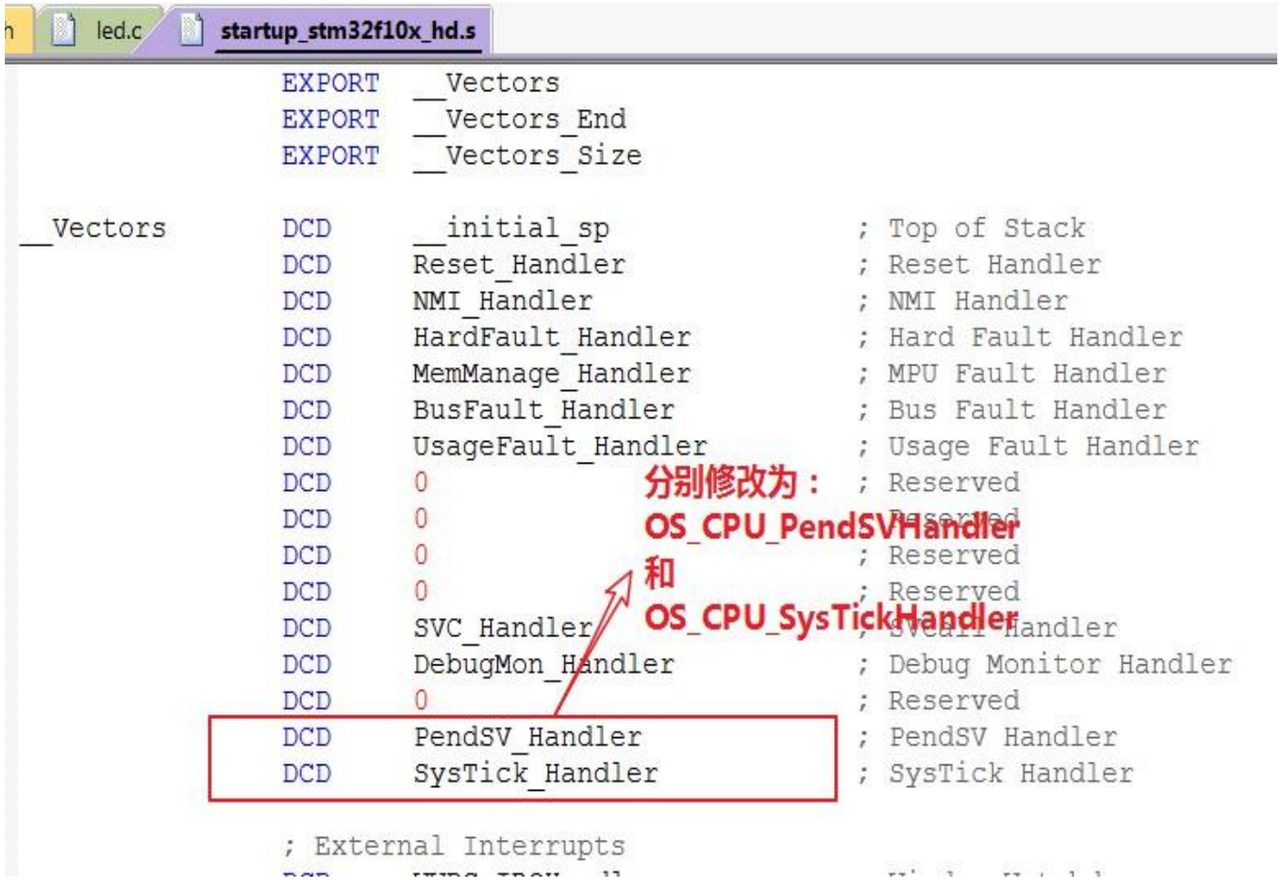
13．添加工程头文件路径，使得编译器能够找到工程所需的“.h”文件。单击魔法棒，选择C/C++，在Include Paths中添加头文件路径。



12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

14．修改工程的启动文件tartup_stm32f10x_hd.s。将其中的“PendSV_Handler”和“SysTick_Handler”修改为“OS_CPU_PendSVHandler”和“OS_CPU_SysTickHandler”，共有三处，如图所示为中断向量表处的修改。



12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

PendSV_Handler和SysTick_Handler函数是STM32的标准外设库所命名的中断处理函数入口名称

不一致

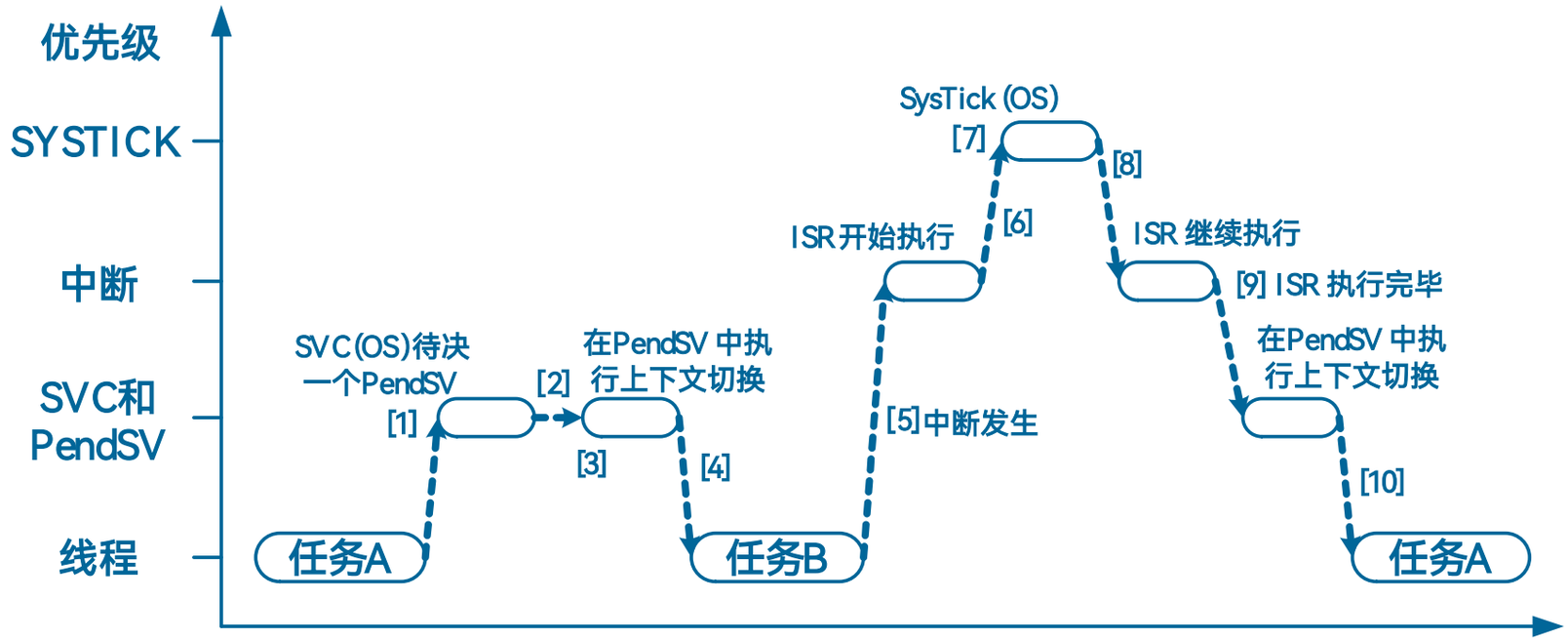
uC/OS-III源码将SysTick_Handler函数命名为OS_CPU_SysTickHandler

必须将启动文件中的PendSV_Handler和SysTick_Handler修改为OS_CPU_PendSVHandler和OS_CPU_SysTickHandler。

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

- SysTick_Handler是滴答定时器中断，为操作系统提供进程/任务的上下文切换和任务调度；
- PendSV_Handler的作用是当操作系统检测到某IRQ正在活动，并且被SysTick抢占时将触发PendSV异常，以便缓期执行上下文切换。



12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

15．修改includes.h文件。

将#include <stm32f10x_lib.h>修改为 #include "stm32f10x.h"。

16．修改bsp.h文件。需要修改的地方有两处：

1）将bsp.h文件中的#include <stm32f10x_lib.h>修改为#include "stm32f10x.h"，并删除bsp_os.h、bsp_ser.h、bsp_i2c.h、bsp_stlm75.h这四个头文件

bsp.h

```
64  #include  <stdio.h>
65
66  #include  <cpu.h>
67  #include  <cpu_core.h>
68
69  #include  <lib_ascii.h>
70  #include  <lib_def.h>
71  #include  <lib_mem.h>
72  #include  <lib_str.h>
73
74  #include  <stm32f10x_lib.h>
75
76  #include  <app_cfg.h>
77
78  #include  <bsp_os.h>
79  #include  <bsp_ser.h>
80  #include  <bsp_i2c.h>
81  #include  <bsp_stlm75.h>
82
83
84  /*
85  ********************************************************************************************************
86  *                                            GPIO PIN DEFINITIONS
87  ********************************************************************************************************
```

修改为 #include "stm32f10x.h"

删除

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

2）删除头文件中#include <app_cfg.h>之下的代码段，只保留BSP_Init (void)和BSP_CPU_ClkFreq(void)两个函数的声明。

bsp.h*

```
55
56
57  /*
58  *********************************************************************************************************
59  *                                              INCLUDE FILES
60  *********************************************************************************************************
61  */
62
63  #include  <stdarg.h>
64  #include  <stdio.h>
65
66  #include  <cpu.h>
67  #include  <cpu_core.h>
68
69  #include  <lib_ascii.h>
70  #include  <lib_def.h>
71  #include  <lib_mem.h>
72  #include  <lib_str.h>
73
74  #include  "stm32f10x.h"
75
76  #include  <app_cfg.h>
77
78
79  void         BSP_Init                       (void);
80
81  CPU_INT32U   BSP_CPU_ClkFreq                (void);
82
83
84  #endif                                                  /* End of module include.
85
```

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

sp.h | bsp.c*

```
#define  BSP_MODULE
#include <bsp.h>

/*
*********************************************************************************************************
*                                             LOCAL CONSTANTS
*********************************************************************************************************
*/

#define  BSP_LED_START_BIT   (13 - 1)                    /* LEDs[3:1] are sequentially connected t

/*
*********************************************************************************************************
*                                          LOCAL GLOBAL VARIABLES
*********************************************************************************************************
*/
CPU_INT32U  BSP_CPU_ClkFreq_MHz;
/*
*********************************************************************************************************
*                                       LOCAL FUNCTION PROTOTYPES
*********************************************************************************************************
*/
static  void  BSP_LED_Init    (void);
static  void  BSP_StatusInit (void);
/*
*********************************************************************************************************
*                                             REGISTERS
*********************************************************************************************************
*/
```

删除

删除

17．修改bsp.c文件。

1）删除#define BSP_LED_START_BIT (13-1)和BSP_LED_Init()、BSP_StatusInit()这些uC/OS-III自带的板载驱动函数

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

17．修改bsp.c文件。

2）将uC/OS-III自带的板载初始化函数BSP_Init (void)内的全部内容删除，改为工程用到的STM32裸机LED初始化函数

```
void  BSP_Init (void)
{
    LED_Init();
}
```

3）删除BSP_LED_Init ()函数及以下的所有以“BSP_”开头的板载驱动函数定义。

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

18．修改app.c文件

1）删除AppTaskCreate ()和AppObjCreate ()这两个函数声明

2）删除BSP_IntDisAll()

```
static  CPU_STK  AppTaskStartStk[APP_TASK_START_STK_SIZE];


/*
*********************************************************************************************************
*                                            FUNCTION PROTOTYPES
*********************************************************************************************************
*/

static  void  AppTaskCreate (void);
static  void  AppObjCreate  (void);
static  void  AppTaskStart  (void *p_arg);
```



12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

18．修改app.c文件

3）删除void AppTaskStart (void *p_arg)中uC/OS-III自带的板载驱动函数及串口

```
static  void  AppTaskStart (void *p_arg)
{
    CPU_INT32U  cpu_clk_freq;
    CPU_INT32U  cnts;
    OS_ERR      err;

   (void)p_arg;

    BSP_Init();                                                 /* Initialize BSP functions
    CPU_Init();

    cpu_clk_freq = BSP_CPU_ClkFreq();                           /* Determine SysTick reference fr
    cnts = cpu_clk_freq / (CPU_INT32U)OSCfg_TickRate_Hz;        /* Determine nbr SysTick incremen
    OS_CPU_SysTickInit(cnts);                                   /* Init uC/OS periodic time src (

    Mem_Init();                                                 /* Initialize Memory Management M

#if OS_CFG_STAT_TASK_EN > 0u
    OSStatTaskCPUUsageInit(&err);                               /* Compute CPU capacity with no t
#endif

    CPU_IntDisMeasMaxCurReset();

#if (APP_CFG_SERIAL_EN == DEF_ENABLED)
    BSP_Ser_Init(115200);                                       /* Enable Serial Interface
#endif

    APP_TRACE_INFO(("Creating Application Tasks...\n\r"));
    AppTaskCreate();                                            /* Create Application Tasks

    APP_TRACE_INFO(("Creating Application Events...\n\r"));
    AppObjCreate();                                             /* Create Application Objects

    BSP_LED_Off(0);

    while (DEF_TRUE) {                                          /* Task body, always written as a
        BSP_LED_Toggle(0);
        OSTimeDlyHMSM(0, 0, 0, 100,
                      OS_OPT_TIME_HMSM_STRICT
```

删除

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

18．修改app.c文件

4）删除AppTaskCreate()和AppObjCreate()这两个空函数

```
*                                          CREATE APPLICATION TASKS
*
* Description:  This function creates the application tasks.
*
* Arguments   :  none
*
* Returns     :  none
*********************************************************************************************************
*/

static  void  AppTaskCreate (void)
{
}

/*
*********************************************************************************************************
*                                          CREATE APPLICATION EVENTS
*
* Description:  This function creates the application kernel objects.
*
* Arguments   :  none
*
* Returns     :  none
*********************************************************************************************************
*/
static  void  AppObjCreate (void)
{
}
```

删除

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

19．编译工程，无错误，无警告。在app.c文件里，添加两个LED灯闪烁的任务函数，验证是否移植成功。

在main()函数之前添加如下代码：

```
#define LED_PRIO 3
#define LED_STK_SIZE 512
static OS_TCB LEDTCB;
static CPU_STK
LEDSTK[LED_STK_SIZE];
static void LED_Task (void *p_arg);
```

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

在AppTaskStart()函数中的CPU_IntDisMeasMaxCurReset()代码之后添加代码：

```
OSTaskCreate((OS_TCB *) &LEDTCB,
             (CPU_CHAR *) "LED",
             (OS_TASK_PTR)LED_Task,
             (void *) 0,
             (OS_PRIO)LED_PRIO,
             (CPU_STK *)&LEDSTK[0],
             (CPU_STK_SIZE)LED_STK_SIZE/10,
             (CPU_STK_SIZE)LED_STK_SIZE,
             (OS_MSG_QTY) 0,
             (OS_TICK) 0,
             (void *)0,
             (OS_OPT)
(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_
CLR),
             (OS_ERR *)&err);
```

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

在AppTaskStart ()函数中的CPU_IntDisMeasMaxCurReset()代码之后添加代码：

```
OSTaskCreate((OS_TCB *) &LEDTCB,
             (CPU_CHAR *) "LED",
             (OS_TASK_PTR)LED_Task,
             (void *) 0,
             (OS_PRIO)LED_PRIO,
             (CPU_STK *)&LEDSTK[0],
             (CPU_STK_SIZE)LED_STK_SIZE/10,
             (CPU_STK_SIZE)LED_STK_SIZE,
             (OS_MSG_QTY) 0,
             (OS_TICK) 0,
             (void *)0,
             (OS_OPT)
(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_C
LR),
             (OS_ERR *)&err);
```

```
static  void  AppTaskStart (void *p_arg)
{
    CPU_INT32U  cpu_clk_freq;
    CPU_INT32U  cnts;
    OS_ERR      err;

   (void)p_arg;

    BSP_Init();                                                 /* Initialize BSP functions
    CPU_Init();

    cpu_clk_freq = BSP_CPU_ClkFreq();                           /* Determine SysTick reference freq.
    cnts = cpu_clk_freq / (CPU_INT32U)OSCfg_TickRate_Hz;        /* Determine nbr SysTick increments
    OS_CPU_SysTickInit(cnts);                                   /* Init uC/OS periodic time src (SysTick).

    Mem_Init();                                                 /* Initialize Memory Management Module

#if OS_CFG_STAT_TASK_EN > 0u
    OSStatTaskCPUUsageInit(&err);                               /* Compute CPU capacity with no task running
#endif

    CPU_IntDisMeasMaxCurReset();

    OSTaskCreate((OS_TCB *) &LEDTCB,
                 (CPU_CHAR *) "LED",
                 (OS_TASK_PTR)LED_Task,
                 (void *) 0,
                 (OS_PRIO)LED_PRIO,
                 (CPU_STK *)&LEDSTK[0],
                 (CPU_STK_SIZE)LED_STK_SIZE/10,
                 (CPU_STK_SIZE)LED_STK_SIZE,
                 (OS_MSG_QTY) 0,
                 (OS_TICK) 0,
                 (void *)0,
                 (OS_OPT) (OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR),
                 (OS_ERR *)&err);
}
```

添加代码

12.4.2 uC/OS-III移植

Step3：针对移植平台，修改相应的源文件

在AppTaskStart()函数后添加如下代码：

```
static void LED_Task (void *p_arg)
{
    OS_ERR err;
    (void )p_arg;
        while(1)
    {       GPIO_SetBits(GPIOB,GPIO_Pin_5);
OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HM
SM_STRICT,&err);
        GPIO_ResetBits(GPIOB,GPIO_Pin_5);

OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HM
SM_STRICT,&err);
    }
}
```

重新编译工程，无错误和警告，则下载到开发板中进行验证。至此，uC/OS-III移植结束。

12.4.3 uC/OS-III多任务应用实例

uC/OS-III实时操作系统能运行多任务，其各种服务都是以任务的形式实现的，uC/OS-III中，每个任务都有唯一的优先级。

基于uC/OS-III应用程序的一般结构如下所示：



12.4.3 uC/OS-III多任务应用实例

```
static void MyTask1 (void
*p_arg)
{
    OS_ERR err;
    (void )p_arg;

    while(1)
    {
        … …
    }
}
//定义用户任务2
static void MyTask2 (void
*p_arg)
{
    OS_ERR err;
    (void )p_arg;
```

```
 while(1)
    {
        … …
    }
}

//定义用户任务3
static void MyTask3 (void
*p_arg)
{
    OS_ERR err;
    (void )p_arg;

    while(1)
    {
        … …
    }
}
```

基于uC/OS-III应用程序的一般结构

```
int  main (void)
{OS_ERR  err;
OSInit(&err);  // 初始化uC/OS-III
BSP_Init();  //硬件初始化
… …
/* 创建用户任务1 */
OSTaskCreate((OS_TCB *)&
MyTask1TCB, (CPU_CHAR
*)"MyTask1",… …);
/* 创建用户任务2*/
OSTaskCreate((OS_TCB *)&
MyTask2TCB, (CPU_CHAR
*)"MyTask2",… …);
/* 创建用户任务3 */
OSTaskCreate((OS_TCB *)&
MyTask3TCB, (CPU_CHAR
*)"MyTask3",… …);
… …
OSStart(&err);
//启动uC/OS-III，开始多任务
```

12.4.3 uC/OS-III多任务应用实例

本应用实例中，在初始任务AppTaskStart中建立MyTask_LED1、MyTask_LED2分别以100ms、300ms的时间间隔控制LED灯闪烁。app.c文件代码如下所示：

```
#include <includes.h>
/*  定义任务优先级  */
#define APP_TASK_START_PRIO 2
#define MyTask_LED1_PRIO 3
#define MyTask_LED2_PRIO 3
/* 设置栈空间大小 */
#define APP_TASK_START_STK_SIZE
128
#define MyTask_LED1_STK_SIZE 512
#define MyTask_LED2_STK_SIZE 512
/* 定义任务控制块 */
static OS_TCB  AppTaskStartTCB;
static OS_TCB  MyTask_LED1TCB;
static OS_TCB  MyTask_LED2TCB;
```

```
/* 定义任务堆栈  */
static CPU_STK
AppTaskStartStk[APP_TASK_START_STK_SIZE];
static CPU_STK
TASK_LED1STK[MyTask_LED1_STK_SIZE];
static CPU_STK
TASK_LED2STK[MyTask_LED2_STK_SIZE];
/* 声明用户任务  */
static void  AppTaskStart  (void *p_arg);
static void  MyTask_LED1 (void *p_arg);
static void  MyTask_LED2 (void *p_arg);
```

12.4.3 uC/OS-III多任务应用实例

```
/* 主函数 main()  */
int  main (void)
{   OS_ERR  err;
    OSInit(&err);  /* 初始化uC/OS-III */
    /* 创建起始任务  */
    OSTaskCreate((OS_TCB  *)&AppTaskStartTCB, // 任务控制块指针，参数类型
OS_TCB  *
    (CPU_CHAR  *)"App Task Start",  //设置任务名称，参数类型CPU_CHAR  *
    (OS_TASK_PTR ) AppTaskStart,  //任务代码指针，参数类型OS_TASK_PTR
    (void *) 0,  //传递给任务的参数p_arg，设置为0表示不使用传递参数
    (OS_PRIO   ) APP_TASK_START_PRIO,  //设置任务优先级
    (CPU_STK  *)&AppTaskStartStk[0],  //设置任务堆栈基地址
    (CPU_STK_SIZE) APP_TASK_START_STK_SIZE / 10,  //设置堆栈剩余警戒线
    (CPU_STK_SIZE) APP_TASK_START_STK_SIZE,  //设置堆栈大小
    (OS_MSG_QTY ) 5u,  // 可接收的最大消息队列数，为0时表示不使用消息队列
    (OS_TICK   ) 0u,  //时间片轮转时间，为0时表示默认长度
    (void      *) 0,  // 任务控制块扩展信息，为0表示不使用任务控制块扩展信息
    (OS_OPT )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR), // 任务选项
    (OS_ERR     *)&err);  // 返回值，调试用
    OSStart(&err);  //启动uC/OS-III
}
```

12.4.3 uC/OS-III多任务应用实例

```
/*  启动任务 STARTUP TASK  */
static  void  AppTaskStart (void *p_arg)
{
CPU_INT32U  cpu_clk_freq;
CPU_INT32U  cnts;
OS_ERR      err;
(void)p_arg;
BSP_Init();  // 板级硬件初始化
CPU_Init();  // CPU初始化
cpu_clk_freq = BSP_CPU_ClkFreq();  //Determine
SysTick reference freq
cnts = cpu_clk_freq /
(CPU_INT32U)OSCfg_TickRate_Hz; // Determine
nbr SysTick increments
OS_CPU_SysTickInit(cnts);  //Init uC/OS periodic
time src (SysTick)
Mem_Init();   // Initialize Memory Management
Module
```

```
#if OS_CFG_STAT_TASK_EN > 0u
   OSStatTaskCPUUsageInit(&err);  // Compute
CPU capacity with no task running
#endif
   CPU_IntDisMeasMaxCurReset();
   OSTaskCreate((OS_TCB *) &MyTask_LED1TCB,
                (CPU_CHAR *) "LED1",
                (OS_TASK_PTR)MyTask_LED1,
                (void *) 0,
                (OS_PRIO)MyTask_LED1_PRIO,
                (CPU_STK *)&TASK_LED1STK[0],

(CPU_STK_SIZE)MyTask_LED1_STK_SIZE/10,

(CPU_STK_SIZE)MyTask_LED1_STK_SIZE,
                (OS_MSG_QTY) 0,
                (OS_TICK) 0,
                (void *)0,
                (OS_OPT)
(OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CL
R),
```

12.4.3 uC/OS-III多任务应用实例

```
OSTaskCreate((OS_TCB *) &MyTask_LED2TCB,
             (CPU_CHAR *) "LED2",
             (OS_TASK_PTR)MyTask_LED2,
             (void *) 0,
             (OS_PRIO)MyTask_LED2_PRIO,
             (CPU_STK *)&TASK_LED2STK[0],
             (CPU_STK_SIZE)MyTask_LED2_STK_SIZE/10,
             (CPU_STK_SIZE)MyTask_LED2_STK_SIZE,
             (OS_MSG_QTY) 0,
             (OS_TICK ) 0,
             (void *) 0,

(OS_OPT)(OS_OPT_TASK_STK_CLR|OS_OPT_TASK_STK_CHK),
             (OS_ERR *) &err);
}
```

12.4.3 uC/OS-III多任务应用实例

```
/*
*用户任务函数名：MyTask_LED1
*LED1任务，优先级为3，以100ms间隔闪烁
*无输入、输出参数
*/
static void MyTask_LED1(void *p_arg)
{   OS_ERR err;
   (void )p_arg;
   while(1)
   {       GPIO_SetBits(GPIOB,GPIO_Pin_5);

OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HMS
M_STRICT,&err);
       GPIO_ResetBits(GPIOB,GPIO_Pin_5);

OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HMS
M_STRICT,&err);
   }
}
```

```
/*
*用户任务函数名：MyTask_LED2
*LED2任务，优先级为3，以300ms间隔闪烁
*无输入、输出参数
*/
static void MyTask_LED2(void *p_arg)
{   OS_ERR err;
   (void )p_arg;
   while(1)
   {      GPIO_SetBits(GPIOE,GPIO_Pin_5);

OSTimeDlyHMSM(0,0,0,300,OS_OPT_TIME_HMS
M_STRICT,&err);
       GPIO_ResetBits(GPIOE,GPIO_Pin_5);

OSTimeDlyHMSM(0,0,0,300,OS_OPT_TIME_HMS
M_STRICT,&err);
   }
}
```

本章小结

概述	嵌入式操作系统的概念	嵌入式实时操作系统uC/OS-III	uC/OS-III移植与应用
• 嵌入式系统的开发方式 • 计算机系统的分层设计思想	• 计算机操作系统 • 嵌入式操作系统 • 嵌入式操作系统的分类	• uC/OS-III的任务 • uC/OS-III时钟和中断 • uC/OS-III任务间通信与同步	• uC/OS-III源码下载 • uC/OS-III移植 • uC/OS-III多任务应用实例



The End!

